

Master's Thesis

Realistic Biome Generation for Procedural Maps using Essential Climate Principles

for obtaining the degree

Master of Science

by

Thomas Würstle

2017-02-28

1. Reviewer: Prof. Dr. Daniel Scherzer
2. Reviewer: Prof. Dr. Martin Zeller

Abstract

This thesis describes a shader based method to generate realistic biome distributions using essential climate principles. It can, for instance, be used to create procedural world maps for global strategy games.

Creating realistic landscapes by simulating climate effects is not a new idea. Comprehensive simulations can be done with existing libraries and services, but they are often more sophisticated than what is needed for games where execution speed is prioritized over accurate emulation. Various guides on how to realize certain aspects of the simulation process can be found on the Internet. However, there are no public resources available that describe the concepts and implementation of a minimalistic yet cohesive method.

Therefore, fundamental climate concepts are summarized and their utilization in the generation process of world maps explained in detail. The presented method is easy to implement, requires no external data and can be computed at runtime. The results are evaluated by comparing them to a map of the Earth. A hexagon-tile based world map is used to demonstrate a possible use case.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Objective	5
1.3	Overview	5
2	Clarification	6
2.1	Reasons for Procedural Generation	6
2.2	Uncanny Valley of Procedural Generation	7
2.3	Similar Works	7
2.4	Climate	10
3	Conception	15
3.1	Approach	15
3.2	Noise based Terrain Generation	16
3.3	Simulation based Terrain Generation	20
3.4	Elevation based Biome Generation	21
3.5	Climate based Biome Generation	23
3.6	Game World Representation	27
4	Implementation	28
4.1	Technology	28
4.2	World Generation	28
4.3	Hexagonal Map	42
4.4	Rendering	45
5	Conclusion	47
5.1	Result	47
5.2	Summary	48
5.3	Outlook	48
	Affidavit	49
	Appendix	50
	Acronyms	51
	Glossary	52
	Figures	60
	Listings	62
	References	63

1 Introduction

This section expresses the motivation behind this project 1.1, states the objective of this work 1.2 and gives an overview of the thesis 1.3.

1.1 Motivation

Reasons for Procedural Generation

Procedural content generation is a common technique in computer games¹ to increase the diversity of their virtual environments and provide different gameplay scenarios when replaying them. Whether it is used to add small features like varying weapon skins or to create the entire game world, its primary purpose lies in reducing the developer's workload so they can focus on exciting gameplay elements. (see section 2.1)

Uncanny Valley of Procedural Generation

When it comes to map generation, most methods solely rely on noise functions based on *pseudo-random number generators* or precalculated *permutation tables*. Due to the repetitive nature of these methods, they lead to recurring structures in the landscape. Since humans excel at pattern recognition², this repetition will be noticed by the player and causes the virtual world to feel unrealistic. (see section 2.2)

Importance of Realism

Unfamiliar settings can be fun to explore, but if a game aims for similarity to the real world, an unrealistic environment or behavior can break immersion. Besides not meeting an expectation, this can impair the player's ability to strategize. For instance, if a player can't relate to the world of a *global strategy game*, he might not be able to decide which geographic features would give him an military advantage, which region he should conquer to gain access to specific resources or which direction he should pursue to find beneficial climate conditions.

Simplicity is Key

Introducing essential principles of Earth's climate into the procedural generation process could overcome the lack of global structure in randomly generated content, thereby escaping the *uncanny valley* and result in more realistic world maps. Sophisticated solutions to realize this already exist³, but lightweight methods, suitable for smaller projects, are currently missing. Hence, this thesis aims to provide one.

¹ Procedural Content Generation Wiki, *Games Featuring Procedural Generation*.

² Mattson, "Superior pattern processing is the essence of the evolved human brain", 2014.

³ See section 2.3.2 *World Engine* and section 2.3.1 *Voxel Farm*.

1.2 Objective

Aim of this project was the derivation of a fast and easy method to produce realistic looking world maps that can be used in computer games, particularly in the global strategy segment. The simulation process should be able to compute at runtime and work without external data to make it suitable for small standalone games.

1.3 Overview

Section 2 [Clarification](#) provides information about procedural content generation, lists similar works and explains essential climate principles.

It describes the benefits of procedural content generation [2.1](#) as well as its potential problems [2.2](#), presents similar works [2.3](#) and explains the essential climate principles [2.4](#) that are relevant for the simulation process (see section [3.1 Approach](#)).

Section 3 [Conception](#) presents the elaborated concepts to incorporate essential climate principles into a procedural world generation process.

It explains the concepts behind the proposed climate simulation approach [3.1](#). After outlining techniques to compute elevation for procedural terrain [3.2](#) and describing the limitations of purely elevation based biomes [3.4](#), two possible climate based methods are presented: approximated linear circulation [3.5.1](#) and pressure based flow circulation [3.5.2](#). The section concludes with an exemplary hexagon tile based world representation [3.6](#) that could be used in a global strategy game.

Section 4 [Implementation](#) describes the realization of the concepts behind the proposed methods for climate based biome generation.

It presents implementations of the concepts to generate and use realistic world maps (see section [3](#)). After explaining the world generation process in detail [4.2](#), the hexagon tile map [4.3](#) as well as the rendering methods [4.4](#) are described.

Section 5 [Conclusion](#) summarizes the thesis, evaluates the results and suggests enhancements for the proposed methods.

It evaluates the result [5.1](#), summarizes the thesis [5.2](#) and gives an outlook on possible improvements [5.3](#) of the presented methods.

2 Clarification

This section describes the benefits of procedural content generation 2.1 as well as its potential problems 2.2, presents similar works 2.3 and explains the essential climate principles 2.4 that are relevant for the simulation process (see section 3.1 Approach).

2.1 Reasons for Procedural Generation

Procedural content generation (PCG) can ...

- ... provide infinite replayability by altering environments and situations on every start of a game.
- ... be adaptive by adjusting to a players capabilities and preferences shown in previous situations.
- ... increase the scope of a game by allowing extensive world sizes with randomly varying content.
- ... reduce memory requirements by storing only the construction rules instead of the content itself.
- ... save development time and costs by lightening the workload of developers, designers and artists.

*"Manual labor has so far ensured that the quality and quantity of game content matched the demands of the playing community, but is facing new scalability challenges due to the exponential growth over the last decade of both the gamer population and the production costs. Procedural Content Generation ... may address these challenges by automating, or aiding in, game content generation."*¹ Hendrikx et al.

The downside of procedural generation is the loss of the ability to craft exactly the experience you want, which is the reason for its rare utilization in narrative games. According to *Extra Credits*, a possible solution to this problem seems to be building core scenarios of the game manually and using procedural content only to fill in the gaps. In their video, that summarizes the pros and cons of procedural generation as well as some common pitfalls, they emphasis: *"Just because the computer is doing the building, doesn't mean you can justify not designing."*² Extra Credits

Dahlskog and Togelius agree, writing: *"In PCG terms, randomization is necessary but not sufficient for automatic level design purposes, as the output of the generator must be controlled in order to fit with the meaning and structure of the game."*³ Dahlskog et al.

¹ Hendrikx et al., "Procedural Content Generation for Games: A Survey", 2013, p. 1.

² Extra Credits, *Procedural Generation - How Games Create Infinite Worlds*, 2015.

³ Dahlskog and Togelius, "Patterns and Procedural Content Generation", 2012, p. 2.

2.2 Uncanny Valley of Procedural Generation

Purely random based procedural generation tends to break *suspension of disbelief* due to a lack of global structure. Similar to the *uncanny valley* of humanoid replicas, it appears as if something is off, even though the environment looks realistic at first. As mentioned in section 1.1, this is caused by the repetitive nature of these generation methods and the excellent pattern recognition capabilities of humans¹.

*"The Perlin noise function, for instance, can be evaluated locally. This means the output of the function depends only on the coordinates of the point, plus some constants. It also means many points can be evaluated in parallel. Since they are isolated from their neighbors, you do not need to compute any neighbors before evaluating a single point. They are blindingly fast, but it comes at a cost: They do not have a soul. They do not produce any information. All what you see comes from a small seed of values and the specific ways these values are churned and shaken by these clever algorithms."*² Cepero

This problem can be solved by introducing global correlations into the generation process, either by synthesizing random structures with existing data from the real world³ or by transferring information between local areas. Climate based world generation aims to accomplish the latter.

2.3 Similar Works

Interesting projects that use climate based world generation are *Voxel Farm*, *World Engine* and *World Project: Ambience*. Games that make extensive use of *procedural content generation* include *Dwarf Fortress*, *Outer Colony*, *Endless Legend* and *Minecraft*.

2.3.1 Voxel Farm

Voxel Farm is a procedural voxel engine that provides sophisticated world generation and allows content modification in a sandbox environment.⁴ Its terrain generation system features climate simulation including plate tectonics and hydraulic erosion and uses *terrain synthesis* to incorporate real world data into the *procedural generation* process.⁵

More information can be found in the *Voxel Farm Dev-Blog: Procedural World*⁶.

¹ Mattson, "Superior pattern processing is the essence of the evolved human brain", 2014.

² Cepero, *Procedural World: Uncanny Valley of Procedural Generation*, 2012.

³ Zhou et al., "Terrain Synthesis from Digital Elevation Models", 2007.

⁴ Voxel Farm, *Voxel Farm - Voxel Engine and Procedural Generation*, 2014.

⁵ Cepero, *Procedural World: Intelligent Terrain Synthesis*, 2016.

⁶ Cepero, *Voxel Farm Dev-Blog: Procedural World*.

2.3.2 World Engine

World Engine is a library that provides a sophisticated world generator. Besides precipitation and temperature its simulation process includes plate tectonics and hydraulic erosion to produce realistic topography. It uses *Holdridge Life Zones* to determine biomes.¹

More information about the concepts behind *World Engine* can be found in *Diving Into Procedural Content Generation, With WorldEngine*².

2.3.3 World Project: Ambience

Ambience is the work-in-progress of a game developed by Dave Frampton. The project features spherical world map generation using climate simulation similar to the method presented in this thesis. It first computes elevation with *Perlin Noise* and calculates temperature based on *latitude*. Afterwards, air pressure is determined depending on surface (land or ocean) and season (summer or winter), which is then used in combination with the *Coriolis Effect* to simulate wind. Finally, precipitation is computed using elevation, temperature and the *Orographic Effect*.³

2.3.4 Dwarf Fortress

Dwarf Fortress is a freeware simulation game with text symbol graphics that uses *procedural content generation* not only for its environment but also its background story. It uses elevation, temperature, rainfall and drainage as factors to determine biomes. Elevation is generated randomly using *midpoint displacement* which produces a fractal map. Temperature is based on elevation and latitude. Rainfall is calculated using *orographic precipitation* and *rain shadow*. Drainage is another randomly generated fractal map that specifies how fast water tickles away. It even calculates salinity, to differentiate between fresh and salt water. The game has been in development since 2002 and launched its first alpha version in 2006.⁴

More information about the world generator of *Dwarf Fortress* can be found in *Interview: The Making of Dwarf Fortress*⁵.

¹ Mindwerks, *World Engine*, 2014.

² Mindwerks, *Diving Into Procedural Content Generation, With WorldEngine*, 2016.

³ Frampton, *Ambience Devblog: Climate*, 2015.

⁴ Wikipedia, *Dwarf Fortress*.

⁵ Harris, *Interview: The Making of Dwarf Fortress*, 2008, p. 7.

2.3.5 Outer Colony

Outer Colony is an open end, real time strategy game with focus on artificial intelligence and simulation, playing in a science fiction setting.¹ Its focus lies on an immersive virtual world, "procedurally generated in such a way as to simulate geological and climatological processes"². Among other things, it uses simulation of geological processes, a simple climate model, *hydrology*, flora, fauna and human activity.

More information about the world generation of *Outer Colony* can be found in *Outer Colony Dev-Blog: Procedural World Generation*³.

2.3.6 Endless Legend

Endless Legend is a turn based 4X fantasy strategy game that uses *procedural content generation* to build its two-dimensional hexagon tile based world map. It features winter seasons that introduce increasingly harder survival conditions as the game progresses. The world generator provides various parameters to influence biome generation, but does not utilize climate based simulation. The game was developed by Amplitude Studios and released in 2014.⁴

2.3.7 Minecraft

The popular sandbox game uses procedural generation to create its world on the fly as the player approaches new areas. Terrain is generated using various noise techniques⁵ and biomes are placed pseudo-randomly with some restrictions to limit neighboring biomes to similar ones⁶. It does, however, not use climate simulation to determine its biomes, since realism is not a priority. The game was made public at a very early stage in 2009 and was officially released in 2011.⁷

¹ Voyager Games, *Outer Colony Dev-Blog: Game Overview*.

² Voyager Games, *Outer Colony Dev-Blog: World Simulation*.

³ Voyager Games, *Outer Colony Dev-Blog: Procedural World Generation*, 2015.

⁴ Amplitude Studios, *Endless Legend*, 2014.

⁵ Persson, *The World of Notch: Terrain Generation*, 2011.

⁶ Bergensten, *Minecraft World Generator Update*, 2013.

⁷ Community, *Minecraft Version History*.

2.4 Climate

This section summarizes essential climate principles: the global circulation patterns [2.4.1](#) as well as the Orographic [2.4.2](#) and Coriolis [2.4.3](#) effects are explained. Additionally, the term biome is defined in regards to nature and games [2.4.4](#).

2.4.1 Global Circulation

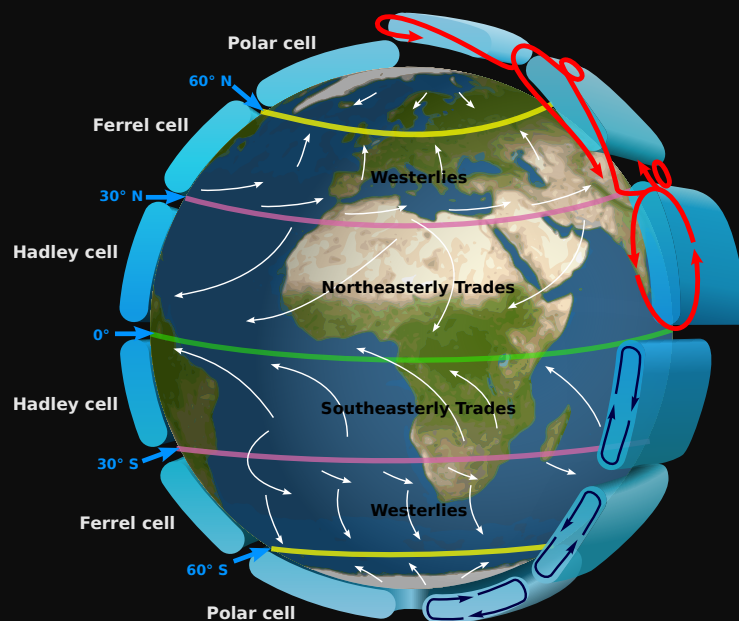


Figure 1: Earth's Global Atmospheric Circulation

Wikipedia, [Earth's Global Circulation](#), 2013

Earth's atmosphere consists of six main circulation cells, three per hemisphere. Each cell surrounds the earth like a belt and stretches over approximately 30° [latitude](#). The strongest two at the equator are called Hadley¹ cells, followed by the Ferrel² cells at the middle [latitudes](#) and finally the Polar cells at the poles.³

The climate is driven by the Sun. Due to the spherical shape of planet Earth, the entrance angle of Sunlight changes with [latitude](#). The sun exposure declines with the angle between the Sun's ray direction and the Earth's surface tangent, since the same amount of sunlight has to cover a wider area of Earth's curved surface. According to the planetary constellation, the Sun mostly hovers above the equator. The resulting difference in solar radiation causes tropical regions to experience an excess of heat while polar regions are left with a deficit.

¹ Named after [George Hadley](#) who proposed a theory to explain the Trade Winds in 1735.

² Named after [William Ferrel](#) who improved Hadley's theory in 1856 by incorporating an overlooked dynamic. They are sometimes called Mid-Latitude cells.

³ Wikipedia, [Atmospheric Circulation](#); Met Office, [Global circulation patterns](#), 2016.

Hadley Cells

The warm air at the equator expands and rises, causing low air pressure at the surface of tropical regions. As it rises, the air cools, due to the atmospheric *lapse rate*. When it reaches the *dew point*, clouds form and rain drains the air's humidity. At the end of the *troposphere*, the *tropopause* acts as a barrier and pushes the air towards the poles. The *Coriolis Effect* deflects its movement to the east, causing high-altitude westerly winds on both hemispheres. At around 30° *latitude*, these westerlies reach maximum velocity and converge into a *jet stream*, preventing the air from traveling further to the poles. This convergence aloft increases the pressure at the surface, causing the dry air to sink slowly. As a result, these conditions produce the *subtropical highs* with generally clear skies and hot surface temperatures, causing vast deserts between 15° and 30° *latitude* on both hemispheres. Due to pressure compensation, most of the surface air then flows back to the *intertropical convergence zone*. On its path, the air is again deflected by the *Coriolis Effect*, resulting in the easterly trade winds. Upon reaching the convergence zone, the air warms and rises again, continuing its cellular circulation.

Ferrel Cells

At the middle *latitudes* on both hemispheres, descending air, that doesn't flow to the equator, is pushed towards the poles. On its way it's deflected towards the east by the *Coriolis Effect*, resulting in a mostly westerly air-flow. In the real world, these *prevailing westerlies* are not constant as they are disturbed by migrating areas of high and low pressure, caused by the topology of the land areas in these *latitudes*. As the air moves towards the poles it cools and the now mild air encounters cold air moving down from the poles. Due to the high difference in temperature, these two air masses can't mix smoothly, causing the warmer air to converge and rise over the cold air, forming a boundary called the *polar front*. This evokes storms, clouds and precipitation and results in a polar *jet stream* aloft as well as a low pressure zone on the surface - the *subpolar low*. Most of the risen air then flows back to the *subtropical highs*, where it sinks and completes its cycle.

Polar Cells

At the *polar front* on both hemispheres, some of the rising air moves towards the poles, where it slowly sinks back to the surface, causing the cold and dry climate in these regions. The resulting high pressure zone, pushes the cooled air outward towards the polar front, where it completes the weak polar circulation.

(This summary is based on *Meteorology today: an introduction to weather, climate, and the environment*¹.)

¹ Ahrens, *Meteorology today: an introduction to weather, climate, and the environment*, 2009, pp. 260-262.

2.4.2 Orographic Effect

This effect causes wet windward slopes and dry leeward slopes as well as dry areas in the rain-shadow of mountains.

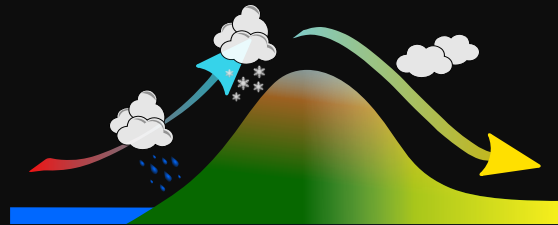


Figure 2: Orographic Effect causes Rain-Shadows

Wikipedia, [Rain Shadow](#), 2007

As the atmosphere moves over land, it follows its topography. Air rises up on mountains and sinks down into valleys. When air is forced upwards over mountains, it cools due to the atmospheric [lapse rate](#). If it reaches the [dew point](#), clouds form and most of the humidity is lost due to precipitation on the windward side of the mountain. After passing the top, the dry air descends on the leeward side of the mountain and warms again. This causes rain shadows - the dry areas on the leeward side of mountain ranges.¹

2.4.3 Coriolis Effect

This effect causes the deflection of moving fluids and gases on both hemispheres of a rotating planet.

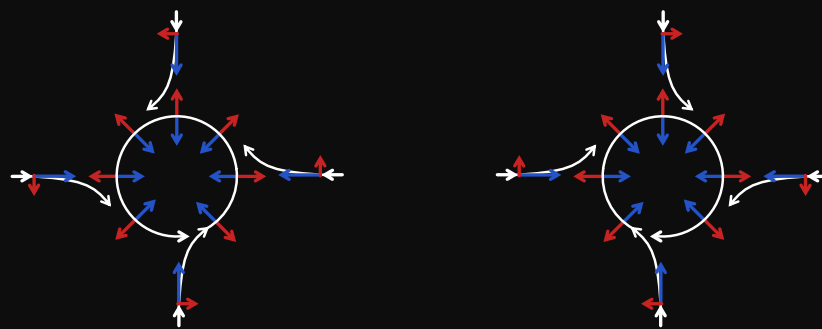


Figure 3: Coriolis Effect on Northern (left) and Southern (right) Hemisphere

"Schematic representation of flow around a low pressure area. Pressure gradient force represented by blue arrows. The Coriolis force, always perpendicular to the velocity, by red arrows."

Wikipedia, [Coriolis Effect](#), 2007

¹ Ahrens, [Meteorology today: an introduction to weather, climate, and the environment](#), 2009, p. 156.

2.4 Climate

As air and water moves over the surface of a rotating planet, they experience a deflection due to the *inertial force* caused by their inertia and angular momentum. In a reference frame with clockwise rotation the force acts to the left, in one with anticlockwise rotation it acts to the right. Since the earth rotates eastward, it causes a deflection to the right on the northern hemisphere and to the left on the southern hemisphere. This prevents winds and currents from flowing straight from areas of high pressure to areas of low pressure.¹

2.4.4 Biomes

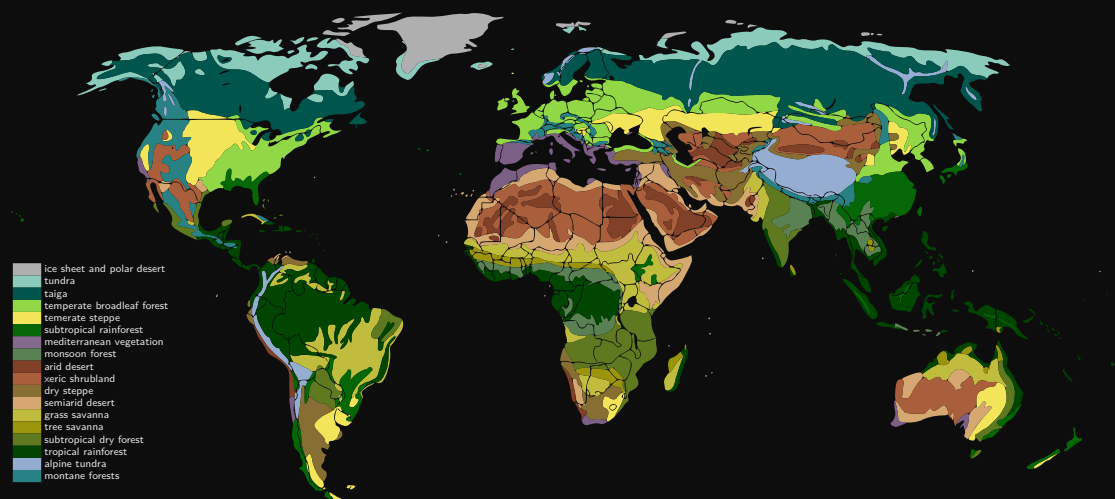


Figure 4: Earth's Global Vegetation Areas

Wikipedia, [Earth's Global Vegetation](#), 2016

Biome Definition

In nature, a biome is a collection of biotic organisms in a certain region defined by its climatic conditions. In strategy games, it is a categorized composition of terrain features like elevation, temperature and precipitation.

"... we can group biological communities and ecosystems into categories based on climate and dominant plant form, which give them their overall character. These categories are referred to as biomes." Ricklefs, [The Economy of Nature](#), 2008, p. 88

"... initial attempts at a world generator will start with things like "I need to lay down some forests, and some mountains, and some rivers, and some deserts..." and then when you end up with a jungle next to a desert, or a desert next to a swamp in an unlikely way, it's difficult to fix. So the idea is to go down to basic elements. The biomes are not the basics, they arise ... from several factors: temperature, rainfall, elevation, drainage ..." Tarn Adams in Harris, [Interview: The Making of Dwarf Fortress](#), 2008, p. 7

¹ Ahrens, [Meteorology today: an introduction to weather, climate, and the environment](#), 2009, p. 206.

Biome Classification

There are many ways to determine biomes based on such factors. A widespread method is the classification defined by Robert Whittaker¹ that only relies on precipitation and temperature.

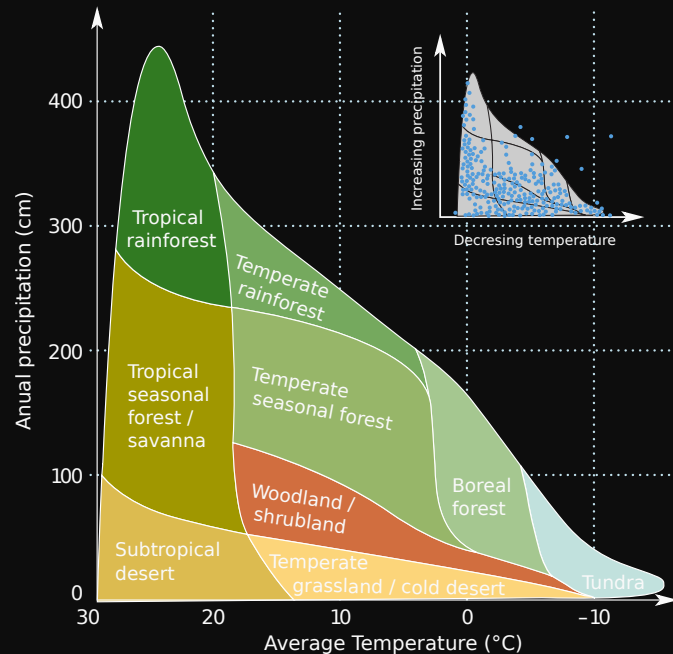


Figure 5: Biomes based on Precipitation and Temperature

"Whittaker plotted the boundaries of observed vegetation types with respect to average temperature and precipitation. ... Inset: Average annual temperature and precipitation for a sample of localities more or less evenly distributed over the land area of the earth. Most of the points fall within a triangular region that includes almost the full range of climates. Only the climates of high mountains do not fall within the triangle." Ricklefs, *The Economy of Nature*, 2008, p. 92

¹ Whittaker, "Classification of Natural Communities", 1962.

3 Conception

This section explains the concepts behind the proposed climate simulation approach [3.1](#). After outlining techniques to compute elevation for procedural terrain [3.2](#) and describing the limitations of purely elevation based biomes [3.4](#), two possible climate based methods are presented: approximated linear circulation [3.5.1](#) and pressure based flow circulation [3.5.2](#). The section concludes with an exemplary hexagon tile based world representation [3.6](#) that could be used in a global strategy game.

3.1 Approach

Reasoning

To make a fictional game world feel realistic, the player needs to be able to relate to it. Therefore, if the virtual world aims to represent a planet similar to Earth, it should resemble the general appearance of Earth's environment. Since playing on an exact replication would soon get boring, methods to change the environment without making it feel unrealistic are required. Fortunately, planets are known to evolve. Over extended periods of time, climate changes and tectonic plates move, altering the appearance in regards to biome distribution and shape of continents. However, fundamental aspects, like the laws of physics, remain constant. Consequently, utilizing essential principles of Earth's climate in procedural world generation might yield realistic results that providing variety in a comprehensive manner.

Ideas

The following steps could incorporate essential climate principles (see [section 2.4](#)) into a procedural world generation process:

1. Generate elevation using fractal random noise. Additionally provide an alternative input method to use existing height maps.
2. Calculate temperature depending on equator-distance and elevation. Take into consideration the heat capacity of land and water.
3. Simulate currents based on global water circulation to determine thermal transfer in oceans.
4. Simulate winds based on global air circulation to determine humidity, precipitation and thermal transfer in the atmosphere.
5. Determine biomes based on temperature, precipitation and elevation.
6. Simulate rivers based on precipitation and elevation to determine erosion. Springs should emerge at close to local maxima with high precipitation.
7. Determine lake positions and sizes according to rivers. Lakes should emerge at local minima with incoming rivers.

3.2 Noise based Terrain Generation



Figure 6: Procedural Noise based Terrain

Shows the positive range of coherent noise using a primary iteration for continents and five additional octaves for mountains and smaller terrain features.

A common method to generate procedural terrain uses *coherent noise* in combination with *Fractional Brownian Motion*.

3.2.1 Coherent Noise

Noise is called coherent if it fulfills the following properties¹:

- Passing in the same input will always return the same output.
- Small changes in the input will produce a small change in the output.
- Large changes in the input will produce a random change in the output.

In combination with a *pseudo-random number generator*, the first property allows recreation of a previously generated world map that is based on a specific seed simply by using the same seed value again.

The latter two properties are important for the generation of elevation since they ensure that the produced data resembles the basic structure of natural terrain. *"In real terrain, heights at different points on the terrain are not independent of each other: the elevation at a specific point on the earth's surface is statistically related to the elevation at nearby points."*²

¹ LibNoise, *What is coherent noise?*

² Shaker, Togelius, and Nelson, *Procedural Content Generation in Games*, 2016, p. 59.

3.2.2 Fractional Brownian Motion

Natural terrain isn't completely smooth. Thus, coherent noise in its basic form will not yield convincing looking results. (see upper left image in figure 7)

To address this issue, a technique based on *Fractional Brownian Motion (FBM)* is used, where additional iterations of noise computation are performed using increasing frequencies and decreasing amplitudes¹. If the frequency doubles in each iteration, they are called *octaves*. The results are combined using a *fractal sum*, which is a weighted sum depending on amplitude.²

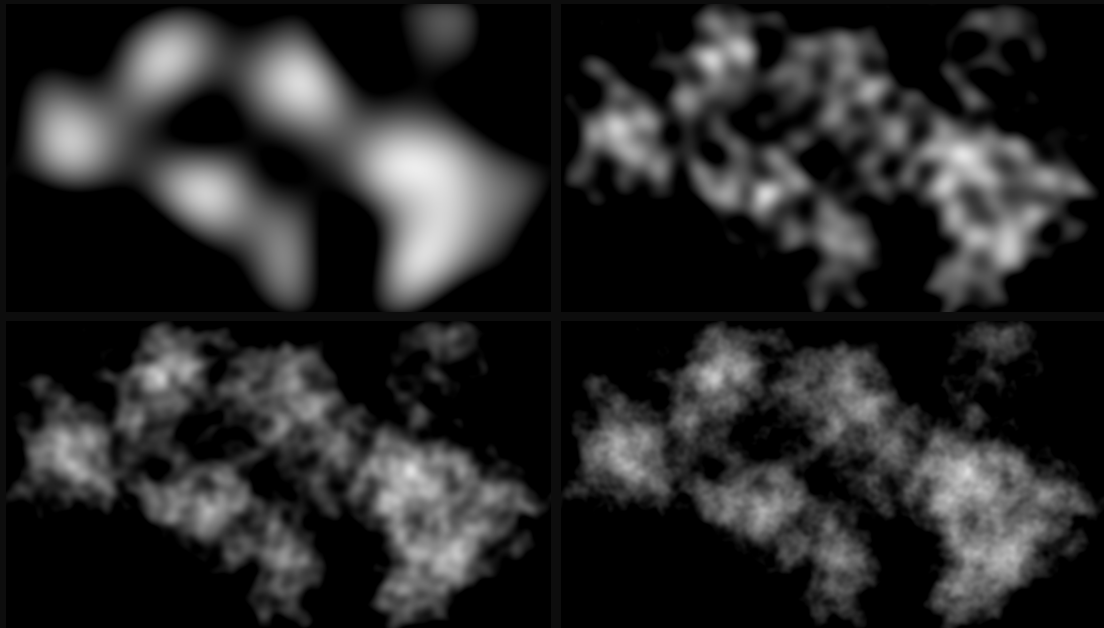


Figure 7: Fractal Coherent Noise

Starting with the upper left and going clockwise, the images show the positive range of coherent noise using a primary iteration in combination with none, one, two and three additional octaves.

$$f_{fbm}(x) = \sum_{i=1}^{\infty} persistence^i \cdot f_n(lacunarity^i \cdot x)$$

f_{fbm} is the function to calculate the *fractal sum* using the noise function f_n . The input x can be either a single number or a vector, depending on the noise dimension. As the iteration progresses, persistence controls the reduction in amplitude and lacunarity the increase in frequency.

Starting the iteration with index one and running until infinity, while using a persistence value equal to one half, would ensure that the sum preserves the original range of possible noise values since the factors would equal the *infinite series*.

¹ An excellent visualization of this aspect can be seen in the video Lague, *Procedural Landmass Generation*, 2016.

² Quílez, *Fractional Brownian Motion Derivatives*.

3.2 Noise based Terrain Generation

However, since infinite iterations are not an option, the sum has to be normalized:

$$f_{fbm}(x) = \frac{\sum_{i=1}^n \text{persistence}^i \cdot f_n(\text{lacunarity}^i \cdot x)}{\sum_{i=1}^n \text{persistence}^i}$$

Normalization preserves the noise range, while allowing the number of iterations, the start index and the persistence value to be chosen arbitrarily.

Using pure octaves means that each iteration doubles the frequency, resulting in terrain that looks more like islands than continents (see left image of figure 8). This is caused by the misrepresentation of continental influence. Continents are dominant geographic features of global terrain, so their impact should differ substantially from that of smaller features like mountains.

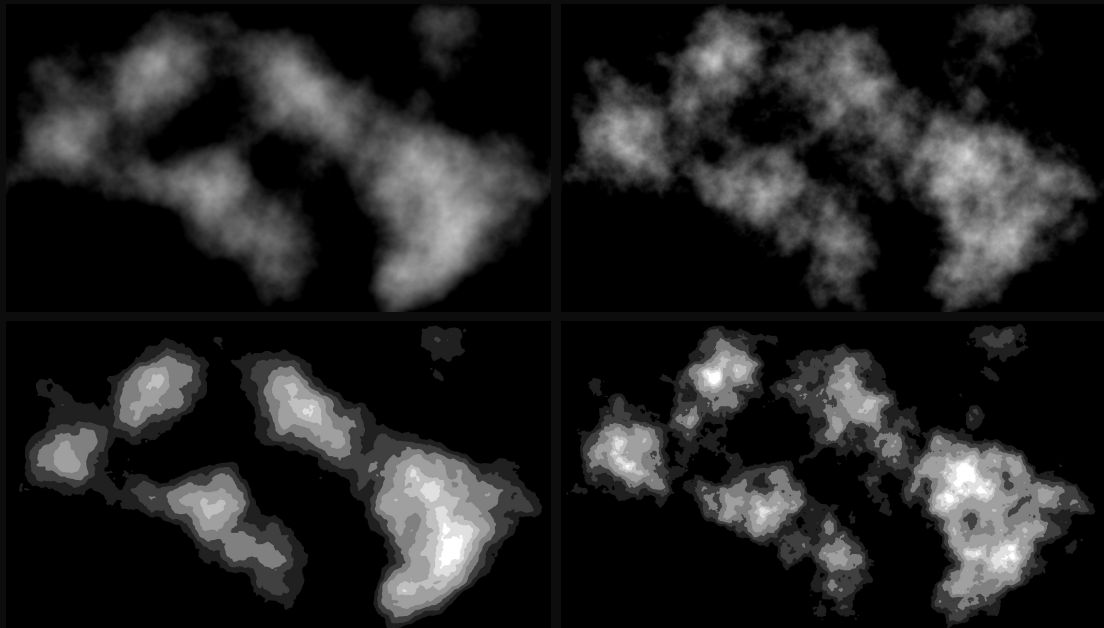


Figure 8: Comparison of Noise based Terrain

Coherent Noise using six octaves (left) versus one primary iteration and five additional octaves (right). Alternatively to the raw noise data in the first row, the second row shows the contour by separating elevation ranges using thresholds. The images show only the positive noise range, meaning they represent land area, if the sea level is set to zero and elevation increases with brightness.

Therefore, a primary iteration is used for continents, that differs in frequency not by the factor of two but four. In combination with five additional octaves, using persistence $\frac{1}{2}$ and lacunarity 2, the formula looks like this:

$$f_{fbm}(x) = \frac{2f_n(\frac{1}{4}x) + f_n(x) + \frac{1}{2}f_n(2x) + \frac{1}{4}f_n(4x) + \frac{1}{8}f_n(8x) + \frac{1}{16}f_n(16x)}{2 + 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16}}$$

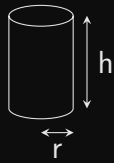
3.2.3 Wrapping Noise

Spherical Wrapping

Since planets are spherical, it is possible to circumnavigate them in any direction. To replicate this property, the elevation map would have to seamlessly repeat itself in all directions. This *spherical wrapping* can be achieved by sampling the two-dimensional surface of a sphere from three-dimensional noise.¹ Spherical world maps can't be displayed in two-dimensions without deformation and therefore require a three-dimensional world representation if distances should be consistent.

Cylindrical Wrapping

However, strategy games often prefer a two-dimensional world map representation that allows an overview of the whole map at once. Such two-dimensional world maps stretch the poles horizontally to cover the entire top and bottom and make them inaccessible. This means the top and bottom borders of these maps can't be exceeded, since this would be equivalent to teleportation from one pole to the other. Continuous movement is therefore limited to the horizontal direction along any fix *latitude*. This *cylindrical wrapping* can be achieved by sampling the two-dimensional mantle of a cylinder from three-dimensional noise.



The presented methods use horizontally wrapping world maps with cylindrical sampling of three-dimensional noise. The cylinder's height h equals the height of the world map, while its circumference equals the width of the world map. The radius r is therefor equal to $\tau \cdot r$.

First, the horizontal component has to be expressed in terms of radians as a fraction of the map's horizontal resolution:

$$\mathbf{N} \rightarrow \mathbf{N} : \quad \mathbf{f}_r(x) = \frac{x}{\text{resolution}} \cdot \tau = \text{radians}$$

Next, the three-dimensional noise coordinates have to be computed based on the position on the two-dimensional world map:

$$\mathbf{N}^2 \rightarrow \mathbf{N}^3 : \quad \mathbf{f}_c(x, y) = \begin{pmatrix} \sin(\mathbf{f}_r(x)) \cdot \text{resolution} \cdot \frac{\text{scaling}}{\tau} \\ y \\ \cos(\mathbf{f}_r(x)) \cdot \text{resolution} \cdot \frac{\text{scaling}}{\tau} \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

Finally, the three-dimensional noise can be sampled for one-dimensional values:

$$\mathbf{N}^2 \rightarrow \mathbf{N} : \quad \mathbf{f}_n(x, y) = \text{noise}_{3D}(\mathbf{f}_c(x, y)) = \text{noise}$$

¹ A detailed explanation can be found in part two of Gallant, *Procedurally Generating Wrapping World Maps*, 2016

3.3 Simulation based Terrain Generation

To achieve even more realistic landscapes, simulation of terrain forming processes like plate tectonics 3.3.1 and hydraulic erosion 3.3.2 would be required. They are utilized in sophisticated solutions like *World Engine* (see section 2.3.2). However, noise based elevation is sufficient for the purpose of this thesis, since its focus lies on generating realistic biome distributions rather than natural terrain structures.

3.3.1 Plate Tectonics

Noise based terrain provides local structure that can be used to form continents, mountains, hills and valleys, but it lacks global structure required to produce mountain chains and ocean trenches (see section 2.2). Achieving more realistic terrain requires the simulation of *plate tectonics*, since this phenomenon forms mountain chains where plates collide and ocean trenches where they drift apart.

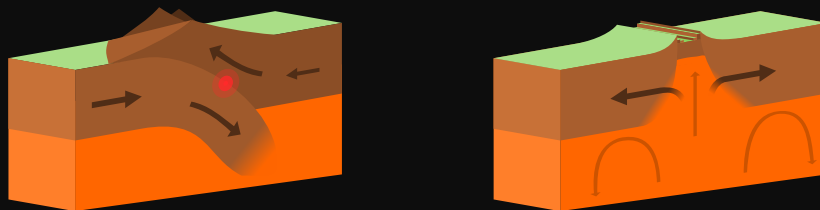


Figure 9: Plate Tectonics

Mountain chains form when plates collide (left) and ocean trenches appear as they drift apart (right).

Wikipedia, [Plate Tectonics](#)

3.3.2 Hydraulic Erosion

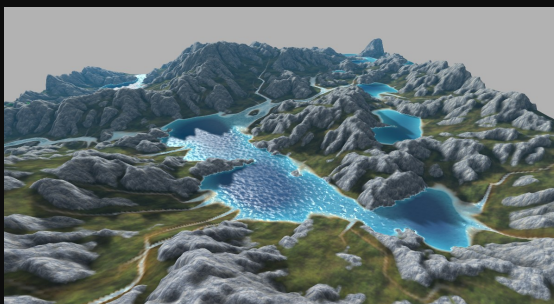


Figure 10: Hydraulic Erosion

Shader based hydraulic erosion.

Boesch, [WebGL GPU Landscaping and Erosion](#), 2011

To produce a realistic balance between soil and rock *hydraulic erosion* is required:

*"Natural mountains and valleys are gradually eroded by rainfall and river flows. Physically-based modeling of this complex phenomenon is a major concern in producing realistic synthesized terrains."*¹

Mei et al.

¹ Mei, Decaudin, and Hu, "Fast Hydraulic Erosion Simulation and Visualization on GPU", 2007.

3.4 Elevation based Biome Generation

There are many ways to generate procedural worlds. A simple, commonly used method uses noise functions to generate elevation and then separates the map by specifying an elevation threshold for each biome.

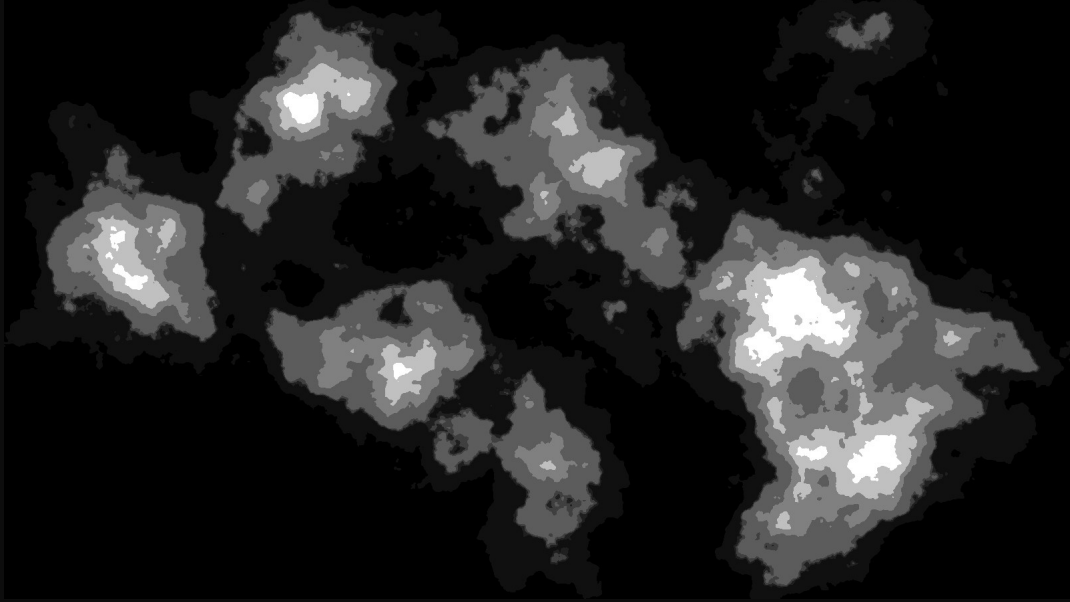


Figure 11: Elevation based Thresholds

The image shows the elevation regions that resulted from grouping elevation values that lie between two thresholds together.

Multiple thresholds are defined and elevation values between them are grouped together and assigned a biome type. For instance, everything below a *sea level* threshold might be *water* and everything above it *shore*, until the next threshold is reached, where *inland* begins.

A possible way to group biomes looks like this:

$$f_{\text{biome}}(\text{elevation}) = \begin{cases} \text{snowcap} & \text{if } \text{mountain threshold} < \text{elevation} \\ \text{mountain} & \text{if } \text{forest threshold} < \text{elevation} \leq \text{snowcap threshold} \\ \text{forest} & \text{if } \text{prairie threshold} < \text{elevation} \leq \text{mountain threshold} \\ \text{prairie} & \text{if } \text{beach threshold} < \text{elevation} \leq \text{forest threshold} \\ \text{beach} & \text{if } \text{coast threshold} < \text{elevation} \leq \text{prairie threshold} \\ \text{coast} & \text{if } \text{ocean threshold} < \text{elevation} \leq \text{beach threshold} \\ \text{ocean} & \text{if } \text{abyssal threshold} < \text{elevation} \leq \text{coast threshold} \\ \text{abyssal} & \text{if } \text{elevation} < \text{ocean threshold} \end{cases} \quad (1)$$

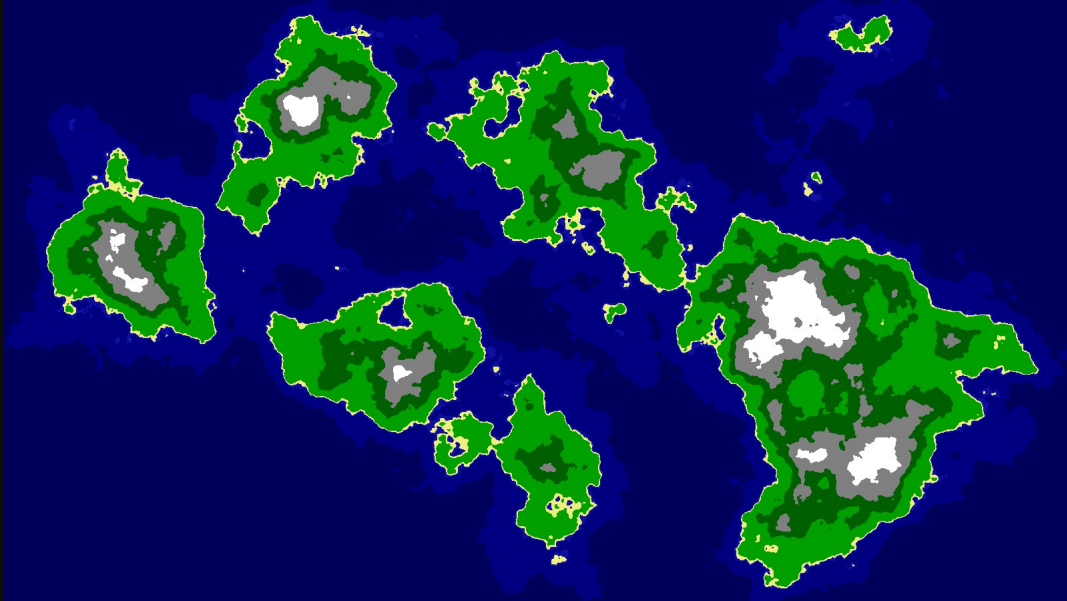


Figure 12: Purely Elevation based Biomes

This method, however, limits biome diversity to generic ones and results in a rather boring map structure due to repetitive patterns. For instance, if biomes are defined like mentioned above (see equation 1), each mountain is surrounded by a forest, which in turn is always surrounded by prairie, meaning there is never a forest or mountain directly next to water (see figures figure 12 and 13). Furthermore, the low biome diversity feels unrealistic due to the lacking variance in temperature and humidity. These problems might be solved by using more noise to generate temperature and humidity maps as well and determining biomes based on a threshold combination of these features.¹ Additional algorithms could increase the diversity and prevent unrealistic combinations of neighboring biomes, but the result would still suffer from recurring patterns caused by the missing global structure of the underlying random generation (see section 2.2).

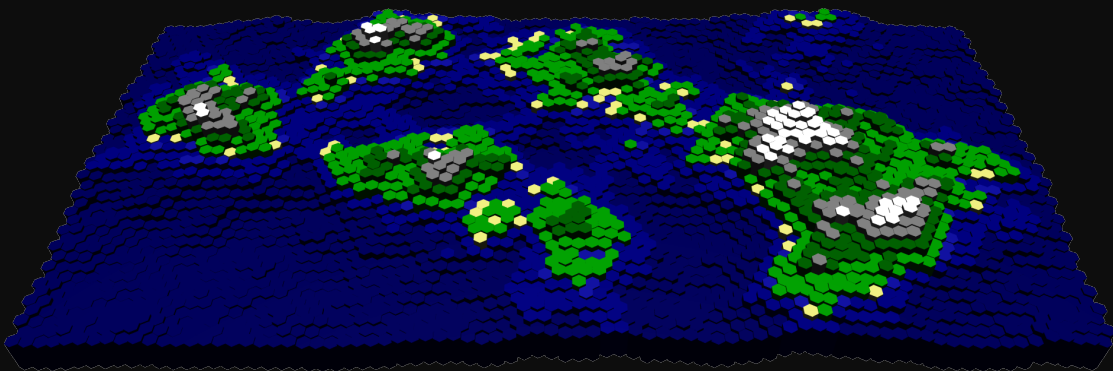


Figure 13: Purely Elevation based World

¹ A detailed explanation can be found in part three of Gallant, *Procedurally Generating Wrapping World Maps*, 2016

3.5 Climate based Biome Generation

3.5.1 Approximated linear Circulation

A minimalistic attempt on climate simulation was elaborated that uses approximated linear distribution of humidity within multiple circulation cells.

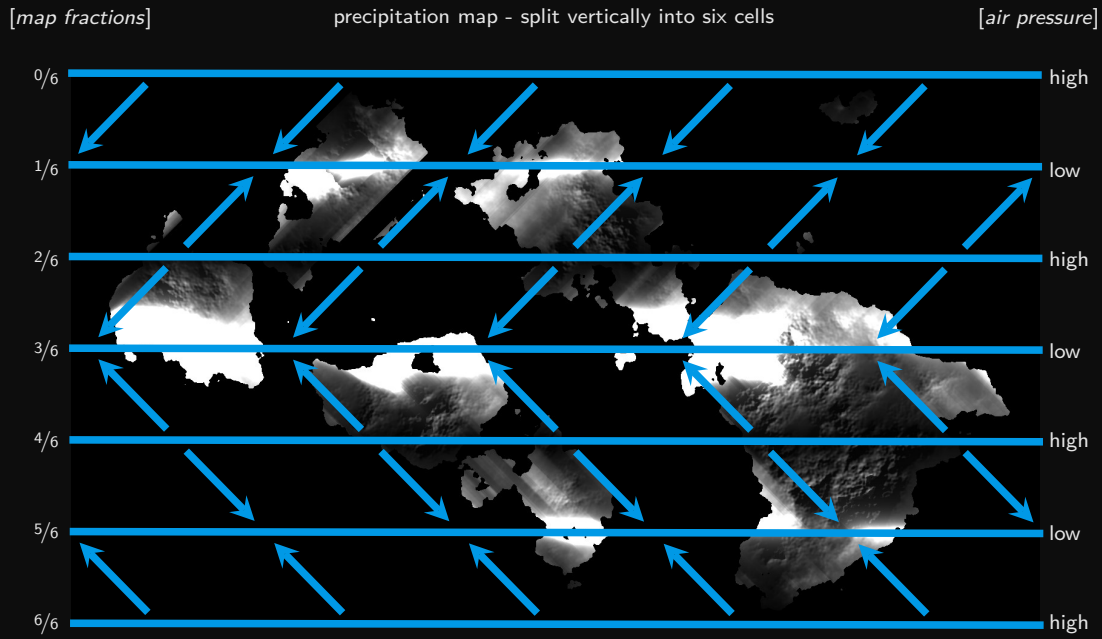


Figure 14: Approximated linear Circulation

The image shows the precipitation that resulted from linear humidity convection within the six circulation cells. The arrows represent the surface wind directions. Aloft wind directions are directly opposite to those on the surface.

Based on global atmospheric circulation cells (see section 2.4.1), the world is divided vertically into six segments (see figure 14). Depending on the general surface wind direction, which includes the *Coriolis Effect*, each cell is assigned a diagonal wind direction (from top to bottom: south-west, north-east, south-west, north-west, south-east, north-west). The simulation then runs a wind based humidity distribution, starting at zones with high surface air pressure at 30° and 90° *latitude* on both hemispheres (which corresponds to 0/6, 2/6, 4/6 and 6/6 of the maps vertical range). Humidity is accumulated over water and released as precipitation over land, depending on the terrain slope in wind direction to emulate the *Orographic Effect*. Excess humidity, that has not been distributed when reaching a circulation boundary is then released in the reverse direction using a quadratic falloff, to emulate precipitation caused by the cooling, rising air and upper winds.

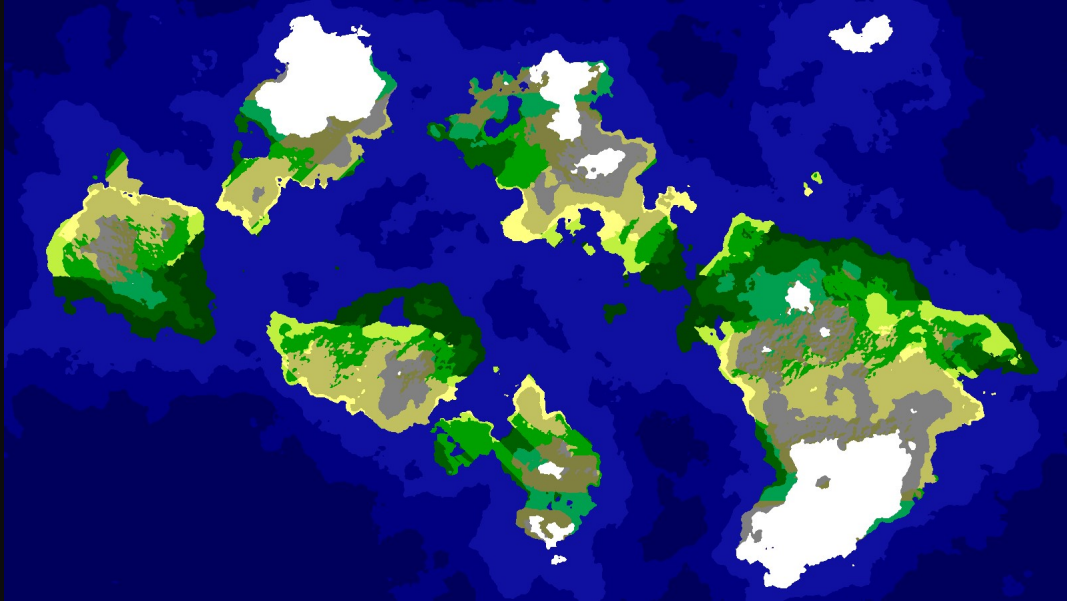


Figure 15: Climate based Biomes using Approximated linear Circulation

This method produces biomes that are more realistic and diverse than those of the elevation based approach (see section 3.4). However, it suffers from sharp edges on the boundaries of the circulation cells, lacking humidity exchange between neighboring cells and linear artifacts due to the missing fluctuations in air flow (see figures 15 and 16).

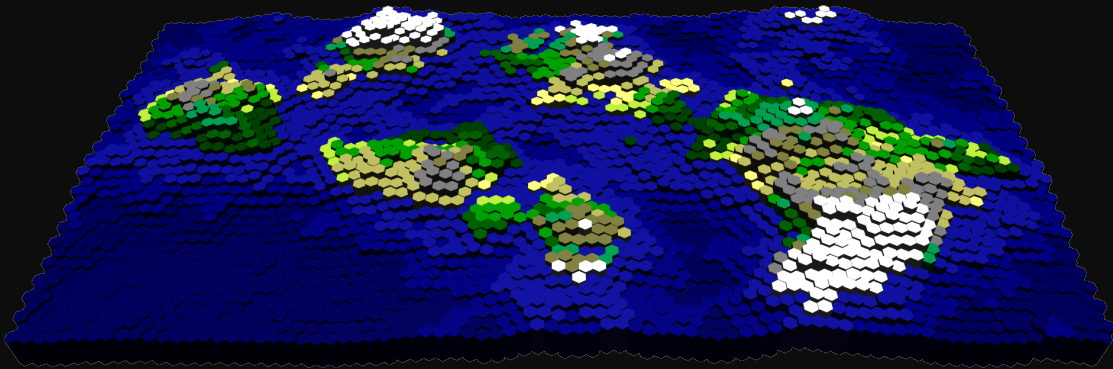


Figure 16: Climate based World using Approximated linear Circulation

3.5.2 Pressure based Circulation

A more sophisticated realization of climate simulation was elaborated that uses an air pressure based flow-field to distribute humidity across the whole map.

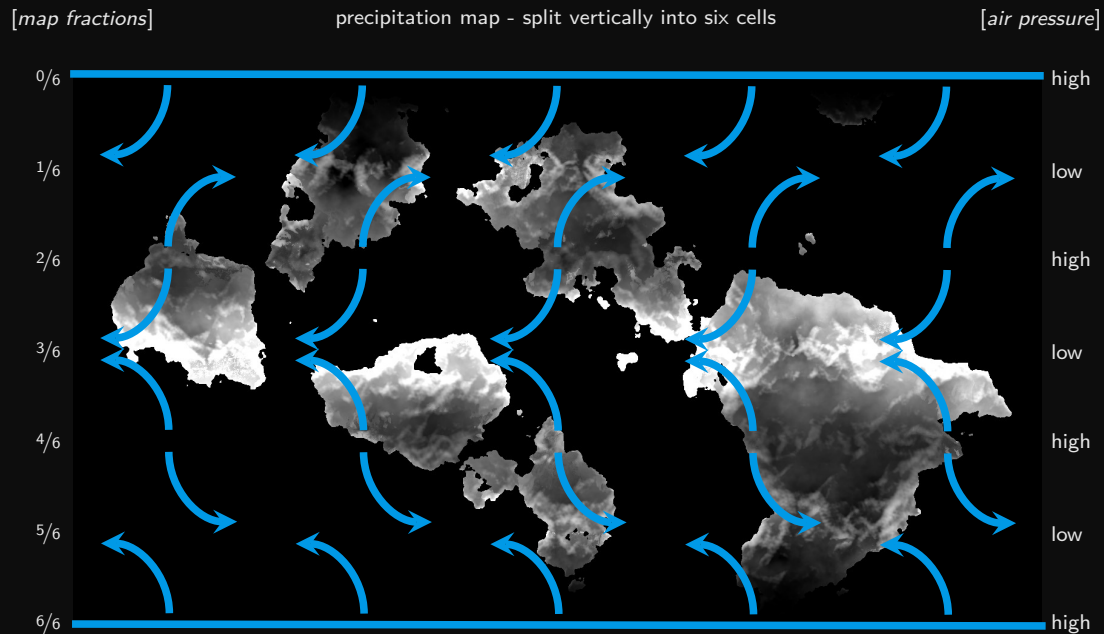


Figure 17: Pressure based Circulation

The image shows the precipitation that resulted from humidity convection according to a pressure based flow-field. The arrows represent the surface wind directions. Aloft wind is not used, since all humidity can be released during the flow circulation.

After generating elevation, temperature is calculated based on elevation and distance from the equator. Subsequently, air pressure is calculated depending on temperature and elevation to determine pressure gradients which then produce wind directions. Additionally, an *evapotranspiration* map is computed based on temperature, wind speed and moisture. Moisture is estimated based on distance from the equator and global circulation patterns (see section 2.4.1 Global Circulation). After that, the humidity circulation is performed using minimalistic fluid simulation¹ that incorporates the *Coriolis Effect*. Finally, precipitation can be determined using humidity and temperature, as well as wind direction and terrain slopes to account for the *Orographic Effect*.

¹ West, "Practical Fluid Mechanics Part I", 2007; West, "Practical Fluid Mechanics Part II", 2007.

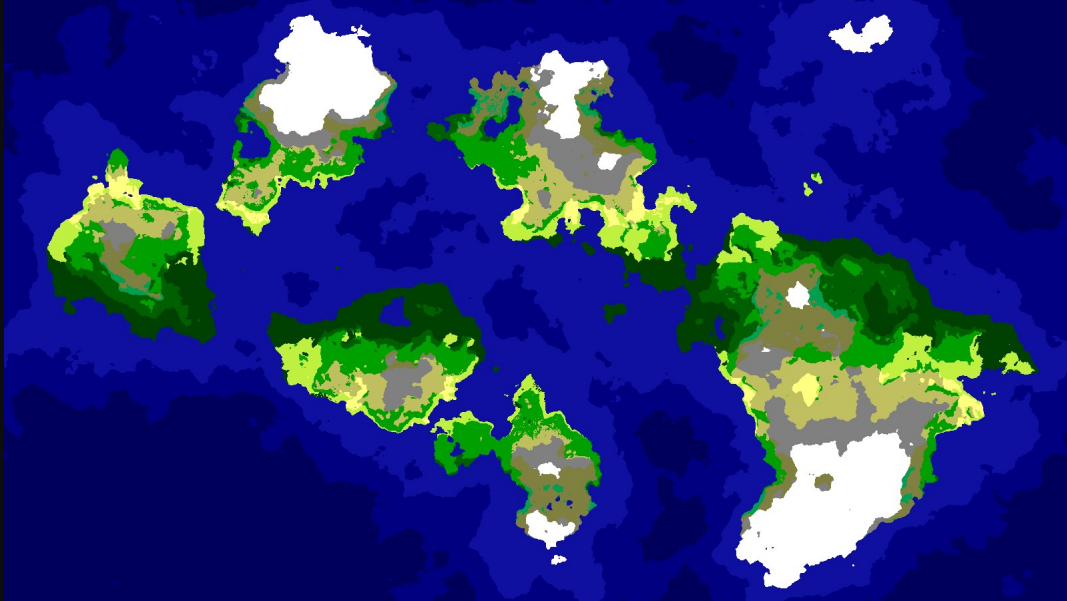


Figure 18: Climate based Biomes using Pressure based Circulation

This results in diverse biomes, distributed similarly to those of the Earth (see figures 18 and 19): wet tropical regions with rain forests, followed on both hemispheres by dry subtropical areas with deserts, moderate mid-latitudes and cold polar regions with icy poles. Additionally, moist windward slopes and *rain shadow* on leeward sides of mountain ranges occur according to global circulation patterns.

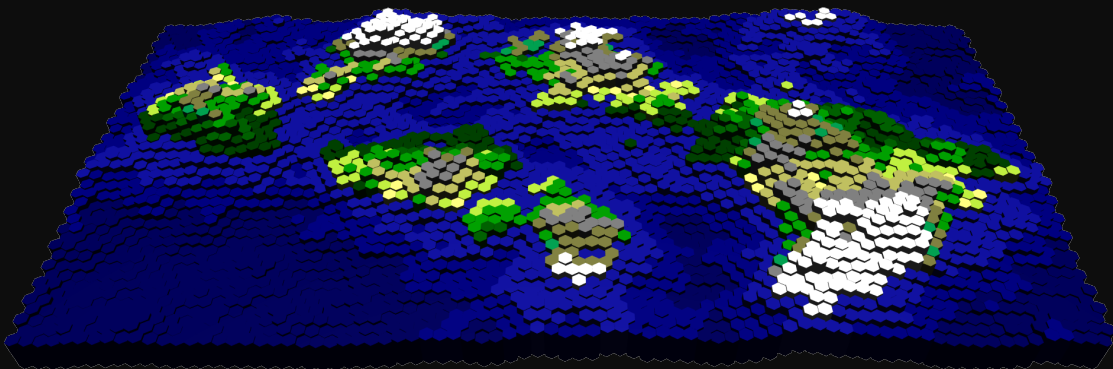


Figure 19: Climate based World using Pressure based Circulation

3.6 Game World Representation

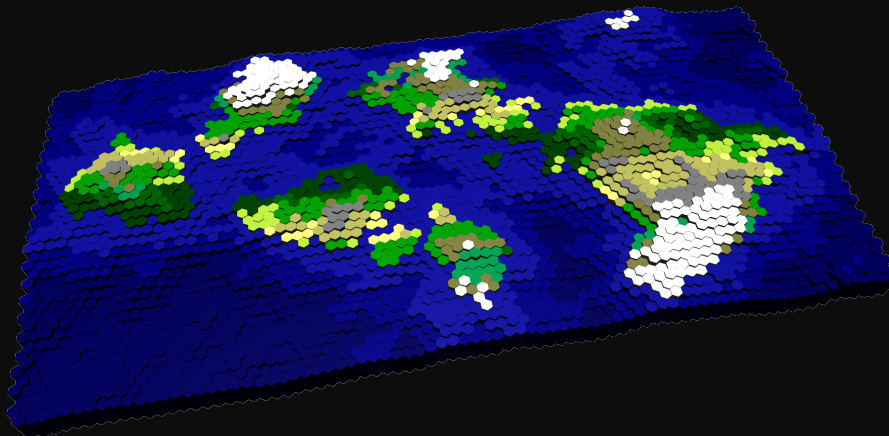


Figure 20: Hexagon Tile based World Map with 80 x 45 Tiles.

Inspired by the game *Endless Legend*¹ (see section 2.3.6), a hexagon tile based world map is used to showcase the results (see figure 20). It represents a global, spherical world, projected onto a two-dimensional world map. Therefore, it wraps horizontally, to allow continuous movement along any fix *latitude*, but not vertically, since this would be equivalent to teleportation from one pole to the other (see section 3.2.3 *Wrapping Noise*).

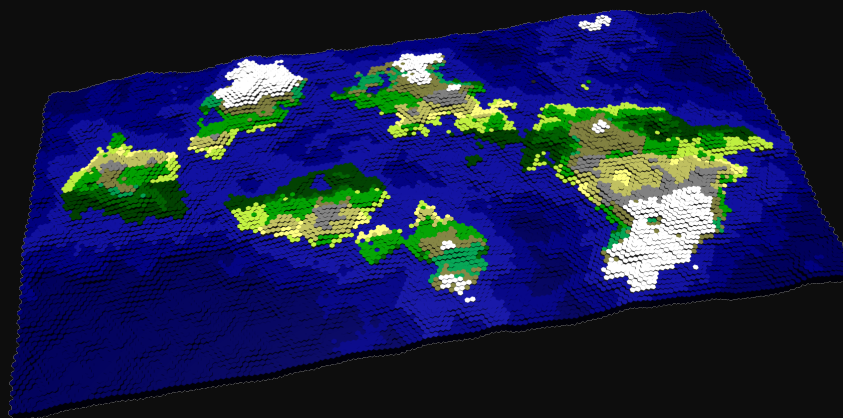


Figure 21: Hexagon Tile based World Map with 160 x 90 Tiles.

The map size can be variable (see figure 21) if the properties of each tile are determined by sampling generated property maps. This means changing the number of tiles that make up the world does not require the world generation process to be executed again. Additionally, this allows the tweaking of properties in retrospect by manipulating the sampled values.

¹ Amplitude Studios, *Endless Legend*, 2014.

4 Implementation

This section presents implementations of the concepts to generate and use realistic world maps (see section 3 [Conception](#)). After explaining the world generation process in detail [4.2](#), the hexagon tile map [4.3](#) as well as the rendering techniques [4.4](#) are described.

4.1 Technology

The project is written in [C++](#) using [Cinder](#) with [OpenGL](#) and [ImGui](#) (see section 4.4 [Rendering](#)). It uses a custom [entity component system](#) and multi-octave [Simplex Noise](#) (see section 4.2.1 [Generating Elevation](#)).

During the experimentation phase the evolving procedure was implemented to work on the [central processing unit \(CPU\)](#) to allow easy restructuring of all execution steps. After that it was rewritten to run on the [graphics processing unit \(GPU\)](#) to make extensive use of parallel computation and thereby improve execution performance.

4.2 World Generation

This section provides a reference implementation of the three described biome generation concepts (see section 3 [Conception](#)): elevation based thresholds [4.2.3](#), approximated linear circulation [4.2.6](#) and pressure based circulation [4.2.11](#). The individual generation steps are explained in detail (see overview in figure [22](#)).

Limitation

The presented implementation doesn't include all ideas listed in section 3.1 [Approach](#): global water circulation (item [3](#)), river simulation (item [6](#)) and river based lake emergence (item [7](#)) were omitted due to time constraints.

Structure

Overview of the implementation structure:

The [CPU](#) based generation steps are implemented using [Cinder's](#) [Surface](#) and [Channel](#) classes. The former is a composition of the latter. A [Channel](#) is essentially a two-dimensional array with a template parameter to specify its data type. [Cinder](#) provides methods to convert [Surfaces](#) and [Channels](#) to textures and vice versa.

The [GPU](#) based generation steps are implemented in fragment shaders. A single point is sent to the vertex shader, which does nothing but passing it to the geometry shader, which turns the point into a square. After that the fragment shader is executed for every pixel of a frame. The later steps receive various textures of previous steps to read from. The temperature shader, for instance, has an elevation map as input.

4.2 World Generation

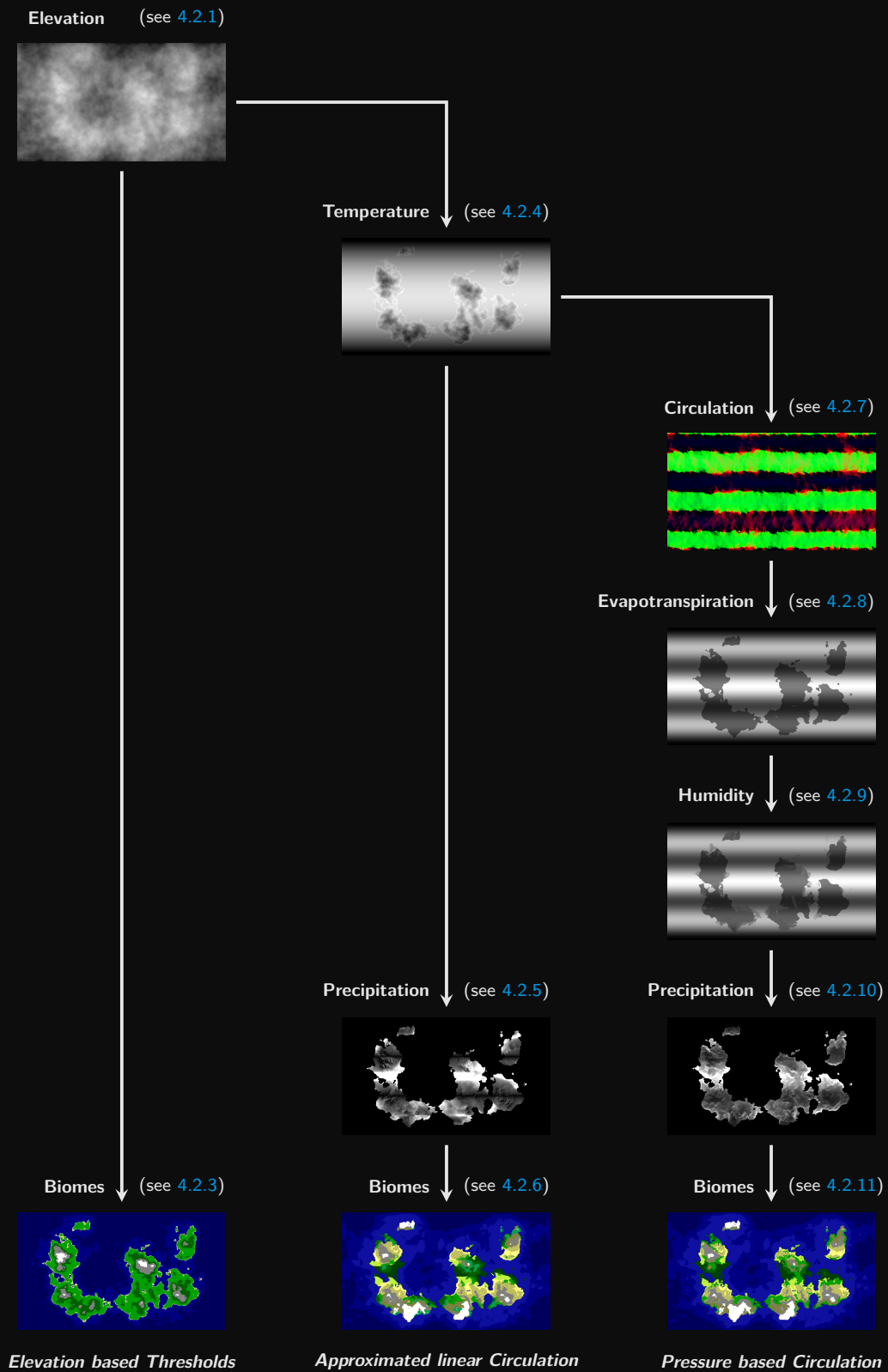


Figure 22: Overview of the Biome Generation Steps

4.2.1 Generating Elevation



Figure 23: Elevation Map, generated with Simplex Noise

Multiple octaves are used to augment a base noise that represents continents with more detail.

Simplex Noise is used to generate an elevation map (see section 3.2 Noise based Terrain Generation) and add some variance during the simulation process. It's a *coherent noise* that is superior to *Perlin Noise* since it runs faster, avoids directional artifacts, has proper derivatives and allows easier generalization for higher dimensions¹. Geilfus's implementation² is used for the *CPU* based simulation while the *GPU* version uses Ashima Arts's *WebGL Simplex Noise* implementation³.

The *Fractional Brownian Motion* aspect (see section 3.2.2) is implemented like this:

```

1 float fractal_noise(vec3 position, int octaves, float lacunarity, float persistence) {
2     float sum = 0.0;
3     float range = 0.0;
4     float frequency = 1.0;
5     float amplitude = 1.0;
6     for (int octave = 1; octave <= octaves; octave++) {
7         sum += simplex_noise(position * frequency) * amplitude;
8         range += amplitude;
9         frequency *= lacunarity;
10        amplitude *= persistence;
11    }
12    return sum / max(range, 1.0);
13 }
```

Listing 1: Fractal Noise

¹ Perlin, "Noise Hardware", 2002.

² Geilfus, *Simplex Noise*.

³ Ashima Arts, *Simplex Noise*.

4.2.2 Composing Elevation

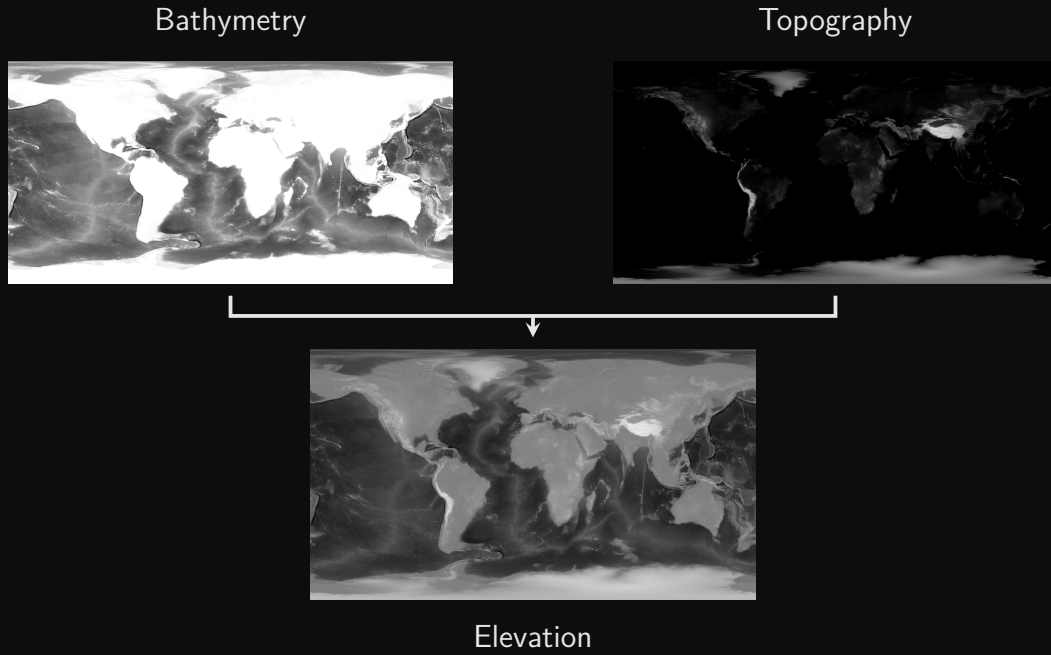


Figure 24: Elevation Map, composed from Bathymetry and Topography

NASA, *Visible Earth: Blue Marble*, 2004

Alternatively, an elevation map can be created using existing height maps. Either by evaluating a single grayscale image containing all height information or by composing one from two images containing bathymetry and topography information.

To provide enough precision and make it consistent with the procedural generated elevation map, a 32 bit floating point texture is used as output. Therefore, bathymetry is mapped onto the negative and topography onto the positive section of the uniform elevation range $[-1.0, +1.0]$.

The generator provides properties to rescale the maps individually and specify a maximum elevation in meters. Scaling bathymetry changes the lower limit of the output range, whereas topography scaling alters the upper limit. The specific value for maximum elevation is used to translate the uniform elevation range to actual elevation data measured in meters which can later be used by a game.

$$\mathbf{N}^2 \rightarrow \mathbf{N} : \quad \mathbf{f}_e(b, t) = (b - 1) \cdot s_b + t \cdot s_t = e$$

$\mathbf{f}_e$ merges bathymetry b and topography t into elevation e and allows individual scaling, assuming
 bathymetry = 1 $\hat{=}$ sea level $\hat{=}$ topography = 0

4.2.3 Determining Biomes based on Elevation

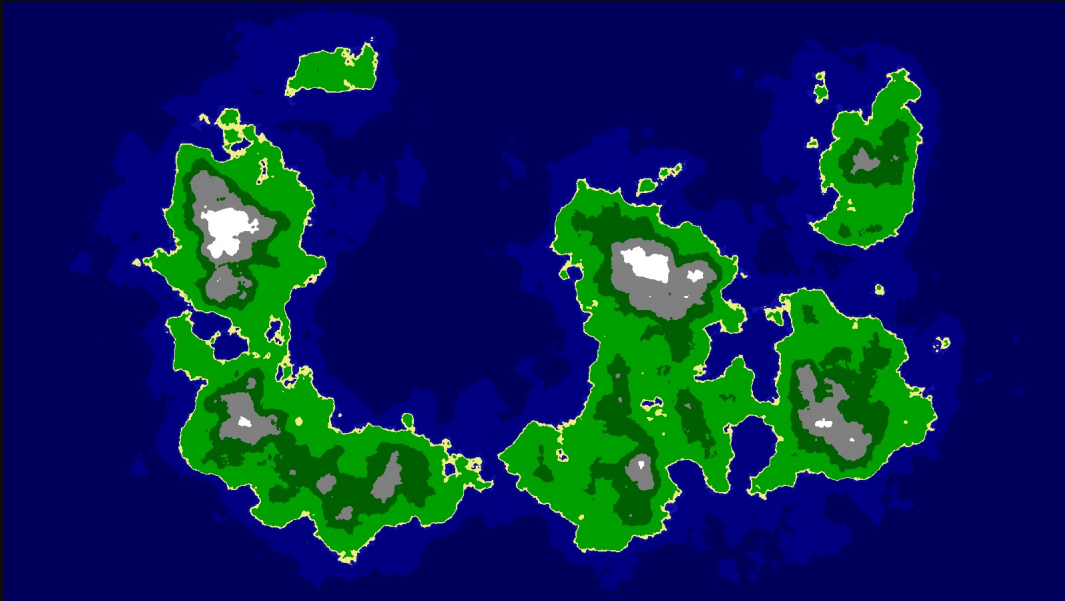


Figure 25: Biome Map based purely on Elevation

Determining biomes purely on elevation [4.2.1](#) means separating the elevation range based on thresholds and assigning a biome to each range:

```

1 Biome determine_biome(float elevation, const BiomeThresholds& thresholds) {
2     if (elevation > 0.33f * sealevel + thresholds.snowcap) return Biome::Snowcap;
3     if (elevation > 0.5f * sealevel + thresholds.mountain) return Biome::Mountain;
4     if (elevation > 0.66f * sealevel + thresholds.forest) return Biome::Forest;
5     if (elevation > sealevel + thresholds.prairie) return Biome::Prairie;
6     if (elevation > sealevel + thresholds.beach) return Biome::Beach;
7     if (elevation > sealevel + thresholds.coast) return Biome::Coast;
8     if (elevation > sealevel + thresholds.ocean) return Biome::Ocean;
9     return Biome::Abyssal;
10 }

```

Listing 2: Elevation based Biome Determination

The hard coded factors for the first three biomes prevent that their thresholds have to be readjusted every time the sea level is changed since they reduce its influence.

Additionally, *flood fill* could be performed to separate ocean and land area, which would allow regions within continents to be dry even if they are below *sea level*. However, this would require the simulation of rivers, possibly using *hydraulic erosion*, to generate lakes, which would render simple elevation based biomes inappropriate by comparison.

4.2.4 Computing Temperature

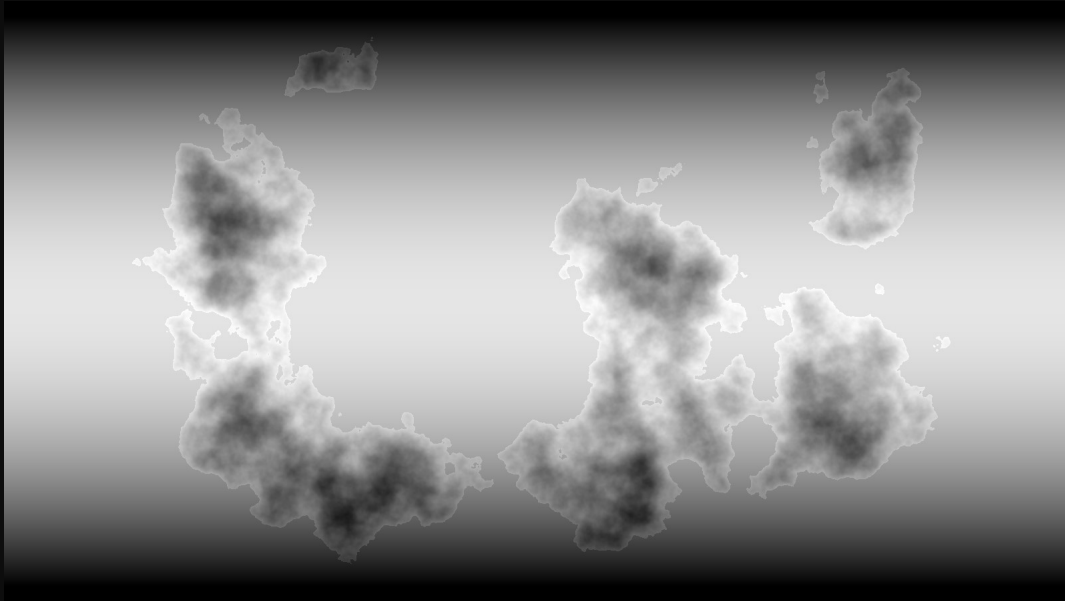


Figure 26: Temperature Map

Temperature is calculated using *equator* distance and elevation [4.2.1](#) based lapse rate.

In the context of the proposed climate simulation methods, the *equator* doesn't represent the geographic equator but rather the *latitude* with highest intensity of solar radiation. Therefore, changing seasons result in a vertically moving *equator*. The distance calculation has to take this into account:

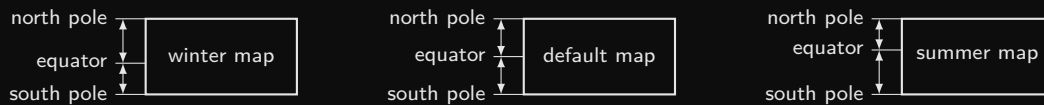


Figure 27: Moving Equator

```

1 float calculate_distance_to_equator() {
2     float latitude = to_signed_range(in_TextureCoordinates.y);
3     float delta = latitude - u_Equator;
4     float range = 1.0 - sign(delta) * u_Equator;
5     return abs(delta) / range;
6 }

```

Listing 3: Normalized Distance to Equator

Calculates distance to the equator as a fraction of the range between the possibly shifted equator and the corresponding pole. The shift is specified by the uniform *u_Equator*.

The function *to_signed_range* is defined in [Appendix](#) listing 10.

Based on this distance, the temperature decreases quadratically towards the poles. Additionally, it declines linearly according to elevation above sea level. The uniform temperature is then scaled using the specified maximum elevation.

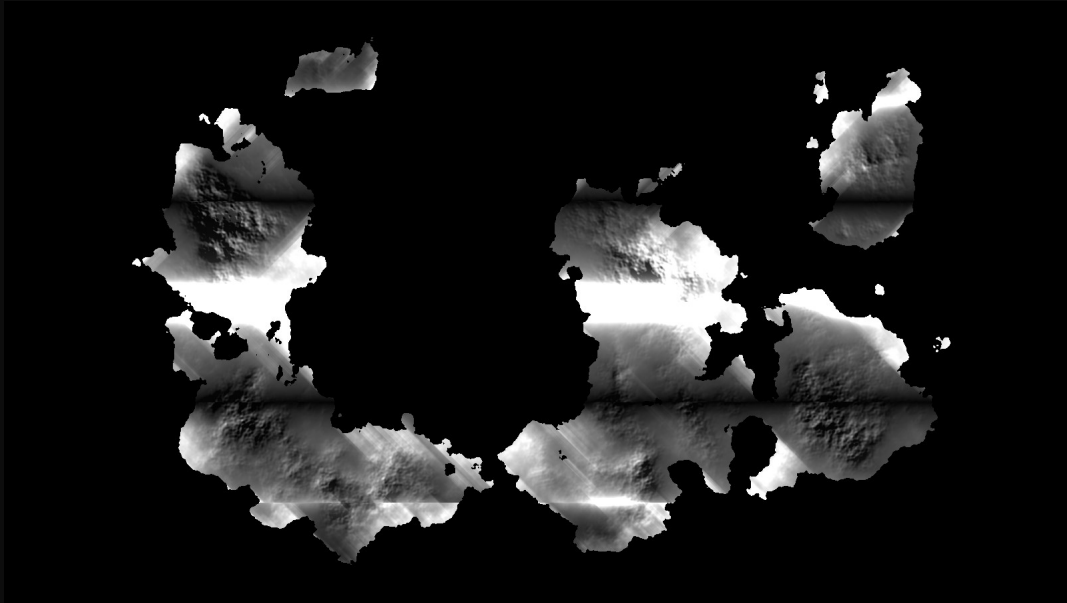
4.2.5 Simulating **Approximated linear Circulation**

Figure 28: Precipitation Map using Approximated linear Circulation

The map is split vertically into six segments, representing the three circulation cells on each hemisphere. To distribute humidity, each segment uses a one-dimensional array with a size equal to the maps horizontal resolution. The algorithm starts at the general high air pressure areas and iterates linearly in a direction, that is approximated based on the deflection caused by the *Coriolis Effect*. During iteration the humidity value is accumulated up to a saturation limit based on a specified *evapotranspiration* rate and turned into precipitation based on its saturation percentage and slope, which accounts for the *Orographic Effect*. Remaining humidity is afterwards released using a quadratic falloff while iterating backwards.

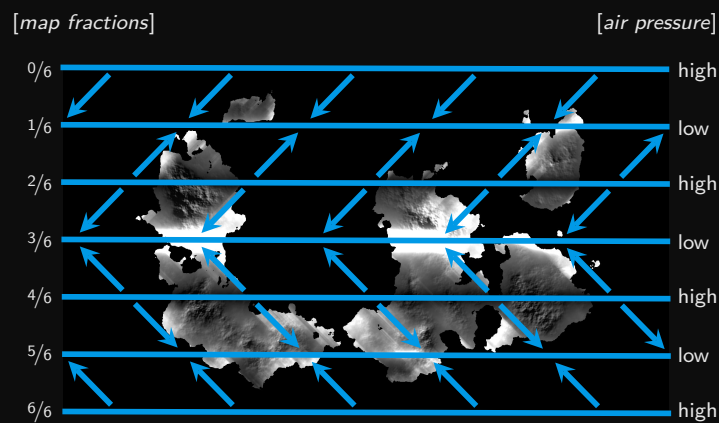


Figure 29: Surface Wind Directions using Approximated linear Circulation.

4.2.6 Determining Biomes based on Approximated linear Circulation

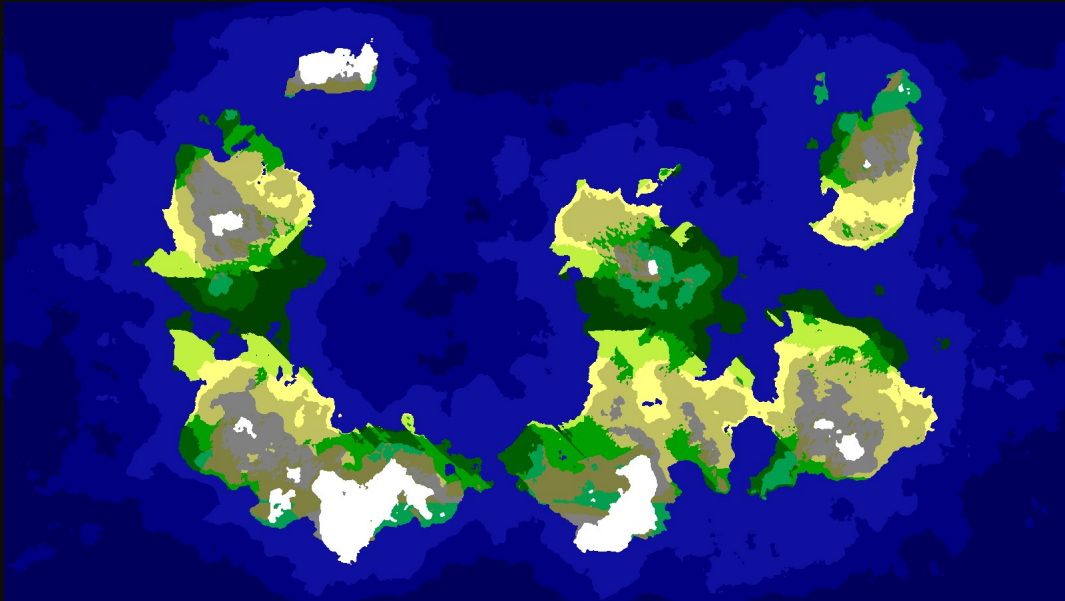


Figure 30: Biome Map using Approximated linear Circulation

Determining biomes using approximated linear circulation means grouping map areas based on three computed parameters: elevation, temperature and precipitation. This is implemented using a lower and an upper threshold that is used for both temperature and precipitation as well as a sea level that separates water from land:

```

1 Biome determine_biome(float elevation, float temperature, float precipitation) {
2     if (elevation < sealevel) {
3         // below sea level from deep to shallow
4         if (elevation < sealevel - 0.5f) return Biome::Abyssal;
5         if (elevation < sealevel - 0.25f) return Biome::Ocean;
6         return Biome::Coast;
7     } else if (temperature < 0.0f) {
8         // below freezing point
9         return Biome::Ice;
10    } else if (temperature < lower_threshold) {
11        // polar regions from dry to wet
12        if (precipitation < lower_threshold) return Biome::Mountain;
13        if (precipitation < upper_threshold) return Biome::Tundra;
14        return Biome::Taiga;
15    } else if (temperature < upper_threshold) {
16        // temperate regions from dry to wet
17        if (precipitation < lower_threshold) return Biome::Steppe;
18        if (precipitation < upper_threshold) return Biome::Prairie;
19        return Biome::Forest;
20    } else {
21        // tropical regions from dry to wet
22        if (precipitation < lower_threshold) return Biome::Desert;
23        if (precipitation < upper_threshold) return Biome::Savanna;
24        return Biome::Rainforest;
25    }
26 }

```

Listing 4: Climate based Biome Determination

4.2.7 Simulating Pressure based Circulation

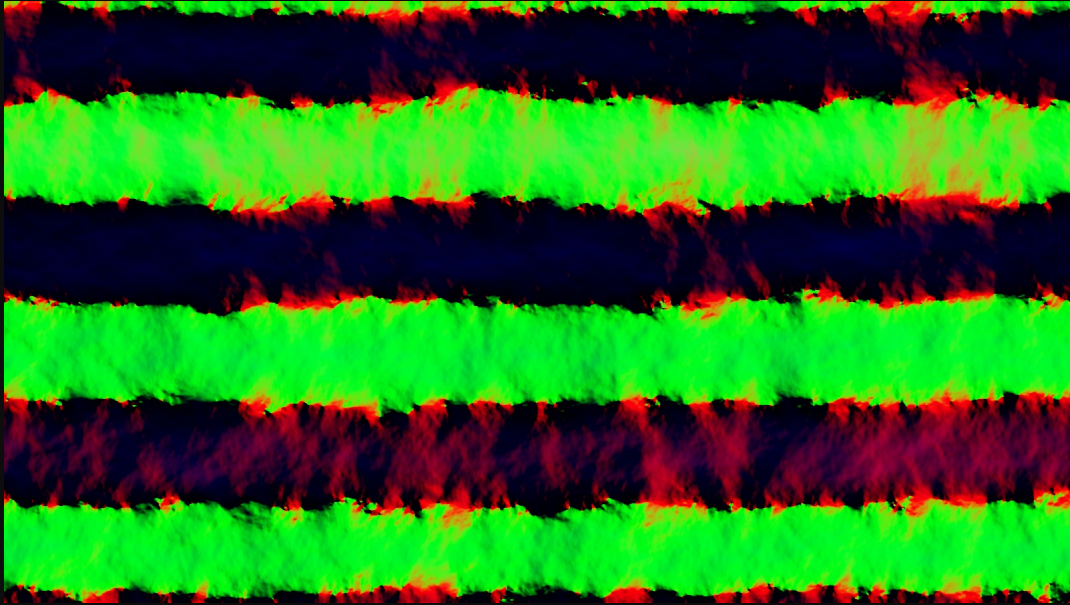


Figure 31: Circulation Map

The red and green channels contain the horizontal and vertical components of the flow direction vector, while the blue channel contains its length that represents flow speed. Due to the condensed information stored in this map, it is difficult to comprehend. (see figure 32 for the debug version)

Circulation calculates air flow using equator distance (see listing 3) and temperature 4.2.4 based air pressure differences.

Air pressure is computed on a per pixel basis by evaluating the difference in air density, which is estimated based on equator distance and temperature (see figure 32). Differences in air pressure are then used to determine wind directions and speed. Finally, the wind directions are deflected according to the *Coriolis Effect* (see section 2.4.3):

```

1 vec2 apply_coriolis_effect(ivec2 texel, vec2 air_flow) {
2     float latitude = float(texel.y) / map_size.y;
3     float equator = to_unsigned_range(u_Equator);
4     float direction = sign(latitude - equator);
5     float flow_speed = length(air_flow);
6     float rotation = rotation_matrix(Tau_Quarter * direction * flow_speed);
7     return rotation * air_flow;
8 }
```

Listing 5: Coriolis Effect

Deflects air flow depending on hemisphere and flow speed.

The function *to_unsigned_range* is defined in Appendix listing 11.

The function *rotation_matrix* is defined in Appendix listing 12

4.2 World Generation

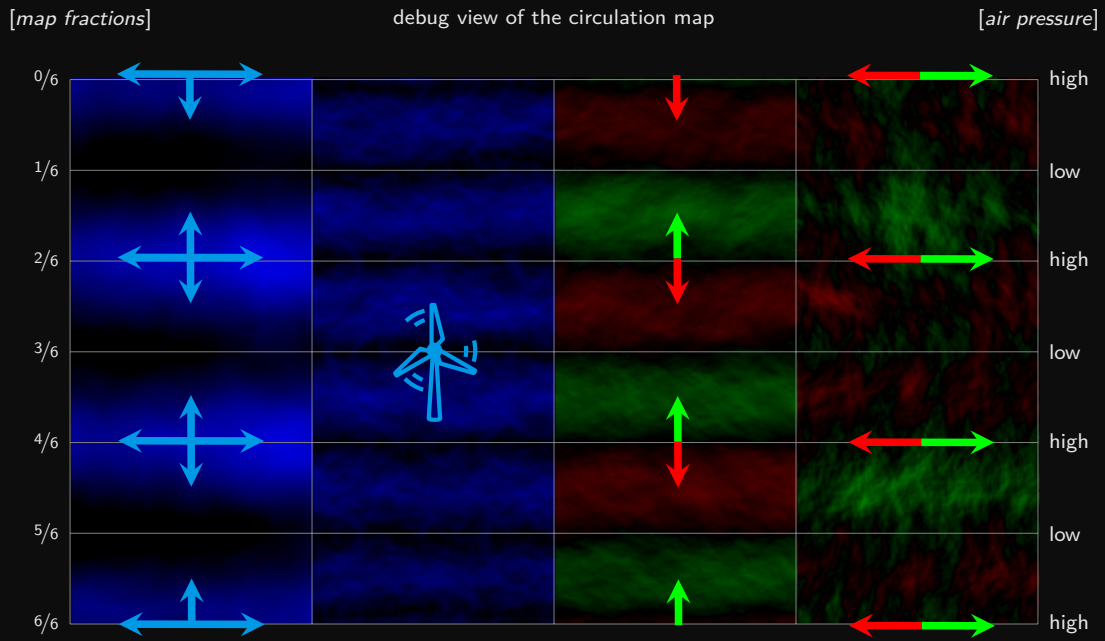


Figure 32: Surface Air Pressure and Flow of Pressure based Circulation

The debug view is split horizontally into four segments to make the components of the circulation flow visible and display air pressure. From left to right: The first segment displays air pressure. The second segment shows the resulting flow speed. The flow direction is displayed in the last two segments: vertical component in the third and horizontal component in the fourth.

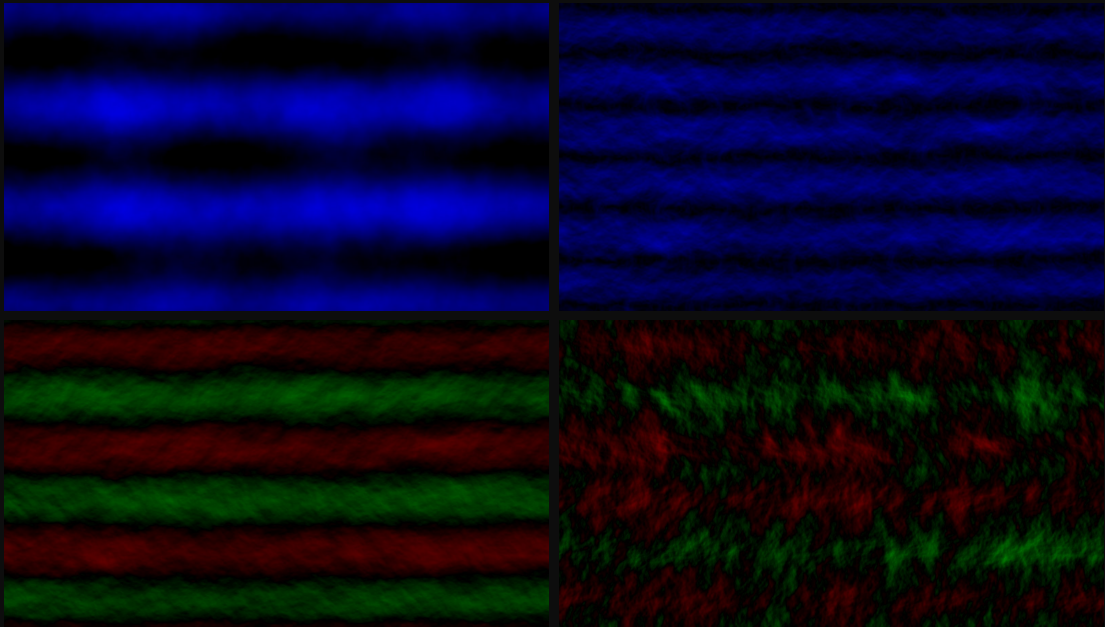


Figure 33: Components of the Circulation Map showing Surface Air Pressure and Flow
Starting with the upper left and going clockwise, the images show air pressure, flow speed, vertical flow and horizontal flow.

4.2.8 Computing Evapotranspiration

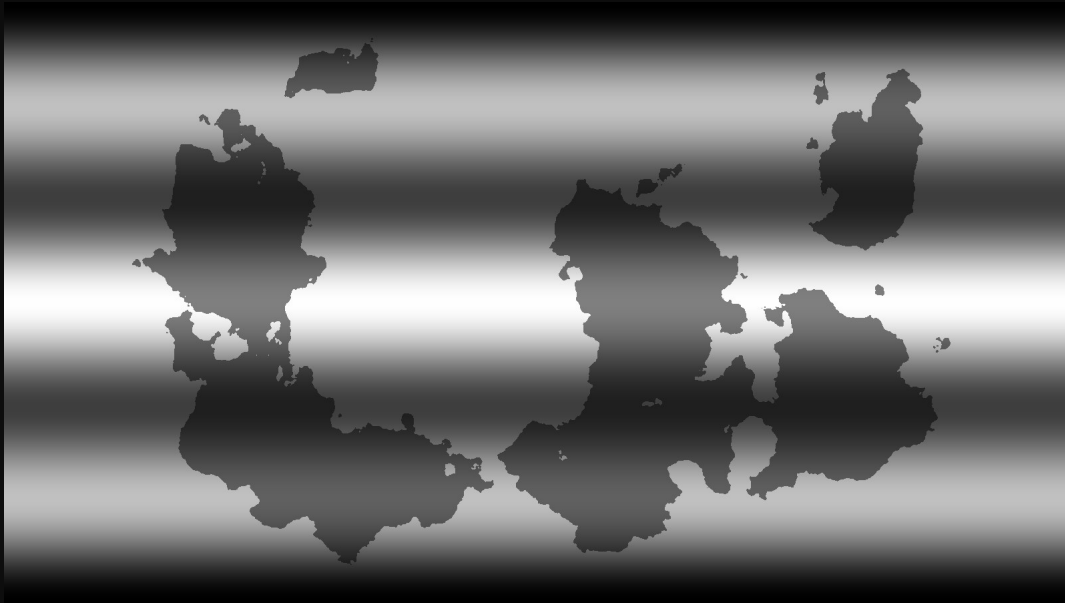


Figure 34: Evapotranspiration Map

Evapotranspiration is calculated using specified water *evaporation* and vegetation *transpiration* rates as well as temperature 4.2.4 and moisture. The latter is estimated based on equator distance:

```
1 float estimate_moisture() {
2     float vertical = to_unsigned_range(calculate_distance_to_equator());
3     return to_unsigned_range(-mix(cos(vertical * 3 * Tau), cos(vertical * Tau), 0.33));
4 }
```

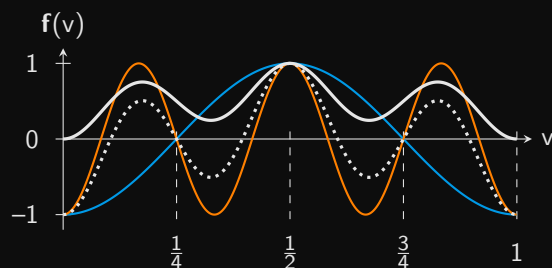
Listing 6: Moisture Estimation

Moisture is estimated based on global atmospheric circulation cells.

The function *to_unsigned_range* is defined in [Appendix listing 11](#).

The function *calculate_distance_to_equator* is defined in [listing 3](#).

This mimics the high moisture that occurs in in the *intertropical convergence zone* as well as the secondary convergence zones at 30° *latitude* on both hemispheres (see [section 2.4.1 Global Circulation](#)).



v is the fraction of the maps vertical range. The white graph represents the estimated moisture, which is constructed using two inverted cosine waves: the orange one specifies three evenly spread convergence zones while the blue one ensures the dominance of the *intertropical convergence zone*.

$$f_s(v) = \frac{1}{3} \cdot -\cos(v \cdot \tau) + \frac{2}{3} \cdot -\cos(v \cdot 3\tau)$$

$$f_u(v) = \frac{1}{2} \cdot f_s(v) + \frac{1}{2}$$

Higher values of moisture and temperature result in a higher value of *evapotranspiration*.

4.2.9 Computing Humidity



Figure 35: Humidity Map

Humidity is based on evapotranspiration 4.2.8 and conveyed by air circulation 4.2.7. It is mainly accumulated over water and distributed over land. The convection process is iterative. It initially takes the *evapotranspiration* map (see figure 34) and performs a convection step according to the wind directions and speeds read from the circulation map (see figure 31). Each successive convection iteration then takes the output of the previous iteration and conveys it further.



Figure 36: Humidity Convection

From left to right: 20, 40 and 60 convection iterations.

During iteration, the slope of the terrain in wind direction is evaluated to determine the *Orographic Effect*, which prevents humidity from traveling over big mountains.

```

1 float orographic_effect(vec3 elevation_gradient, vec2 wind_direction, bool land) {
2     float slope = land? 1.0 - dot(elevation_gradient, vec3(0,1,0)) : 0.0;
3     float uphill = max(dot(wind_direction, -elevation_gradient.xz), 0.0);
4     return uphill * slope * u_OrographicEffect;
5 }

```

Listing 7: Orographic Effect

Calculates the dot product between wind direction and the elevation gradient's horizontal projection.

4.2.10 Computing Precipitation

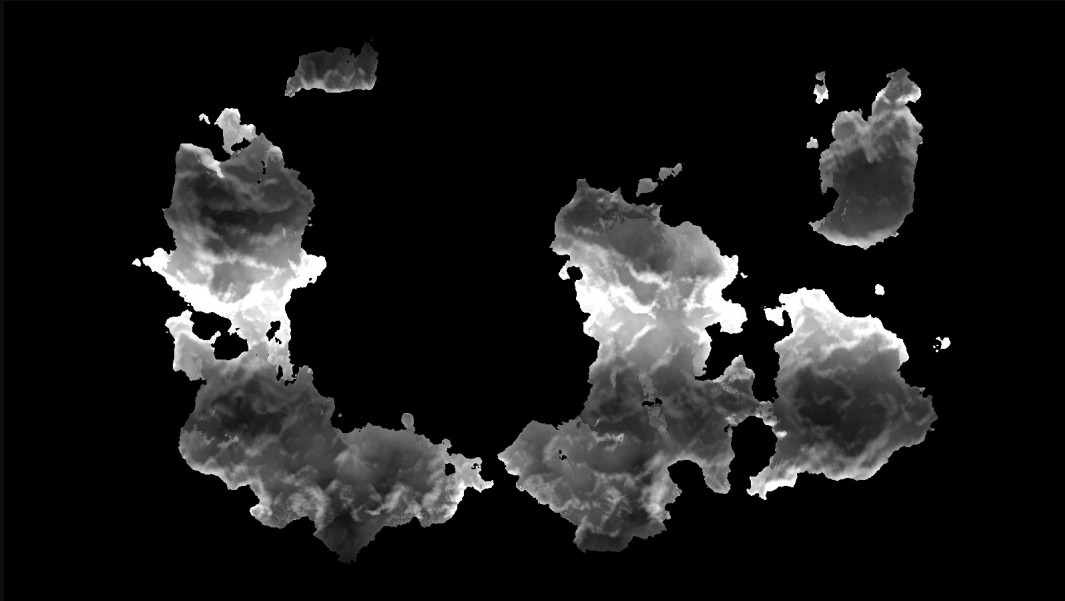


Figure 37: Precipitation Map

Precipitation is computed using elevation [4.2.1](#), temperature [4.2.4](#), air flow [4.2.7](#), moisture [4.2.8](#) and humidity [4.2.9](#).

This is the final circulation step that combines all previously computed properties to produce realistic precipitation patterns. The implementation looks like this:

```

1 float estimate_base_precipitation() {
2     float vertical = to_unsigned_range(calculate_distance_to_equator());
3     return to_unsigned_range(-cos(vertical * 3 * Tau));
4 }
5
6 float calculate_precipitation(float humidity, float thermal, float orographic) {
7     float estimated = (1.0 - u_Circulation) * estimate_base_precipitation();
8     float simulated = (2.0 * u_Circulation) * (thermal + orographic) * humidity;
9     return u_Intensity * (estimated + simulated);
10 }
```

Listing 8: Circulation based Precipitation

The function *estimate_base_precipitation* works like listing [6 Moisture Estimation](#) but treats every atmospheric convergence zone equally to emulate the intense precipitation that occurs in these areas (see section [2.4.1 Global Circulation](#)). The result is then combined with the conveyed *humidity* using *thermal* and *orographic* factors calculated in previous steps. While the uniform *u_Circulation* controls the balance between estimated and simulated precipitation, *u_Intensity* allows the manipulation of the global precipitation intensity.

The function *to_unsigned_range* is defined in [Appendix listing 11](#).
The function *calculate_distance_to_equator* is defined in [listing 3](#).

4.2.11 Determining Biomes based on [Pressure based Circulation](#)

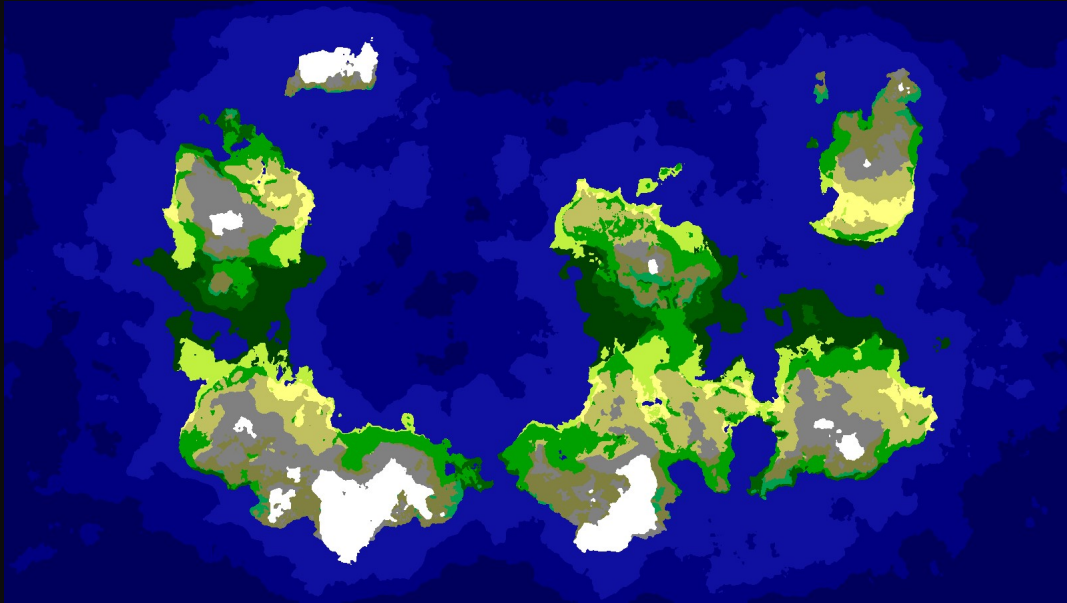


Figure 38: Biome Map using Pressure based Circulation

Biome determination can either be realized by grouping according to threshold combinations (see [listing 4](#)) or by sampling a classification map (see [figure 5](#)).

The current version uses threshold based biome determination, to allow a direct comparison of the two climate simulation methods [Approximated linear Circulation](#) and [Pressure based Circulation](#).

Alternatively, the sampling method could be implemented by turning a classification image (like [figure 5](#)) into a map, that contains only the colored regions. A fragment shader, that reads precipitation and temperature from an input texture, could then sample the classification map at the corresponding position ($x=\text{temperature}$, $y=\text{precipitation}$) and write the sampled color to an output texture (similar to [figure 38](#)). The resulting biome map can then be sampled for any world position (see [section 3.6 Game World Representation](#)).

4.3 Hexagonal Map

4.3.1 Tiles

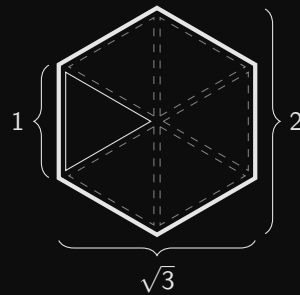


Figure 39: Hexagon Tile

The tiles are regular pointy hexagons. Regular means that all six sides have the same length, which allows it to be constructed using six equilateral triangles (see figure 39). Setting the side length to 1 results in a hexagon with a width of $\sqrt{3}$ and a height of 2. The layout is specified by *pointy*, which states that the hexagon is standing on one of its corners instead of lying *flat* on an edge (see figure 40).



Figure 40: Hexagon Tile Types

Pointy hexagons were used since they allow uninterrupted horizontal movement (see figure 41), which is preferable for horizontal wrapping maps (see section 3.6 Game World Representation).

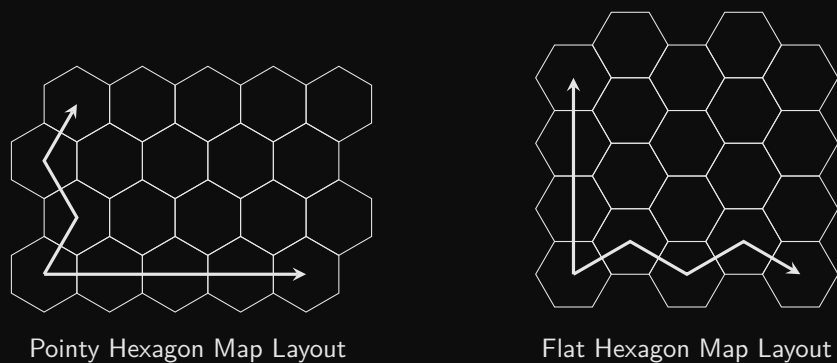


Figure 41: Hexagon Map Movement

4.3.2 Coordinates

The hexagonal grid uses homogeneous coordinates with the constraint:

$$u + v + w = 0$$

Its axes and coordinates look like this:

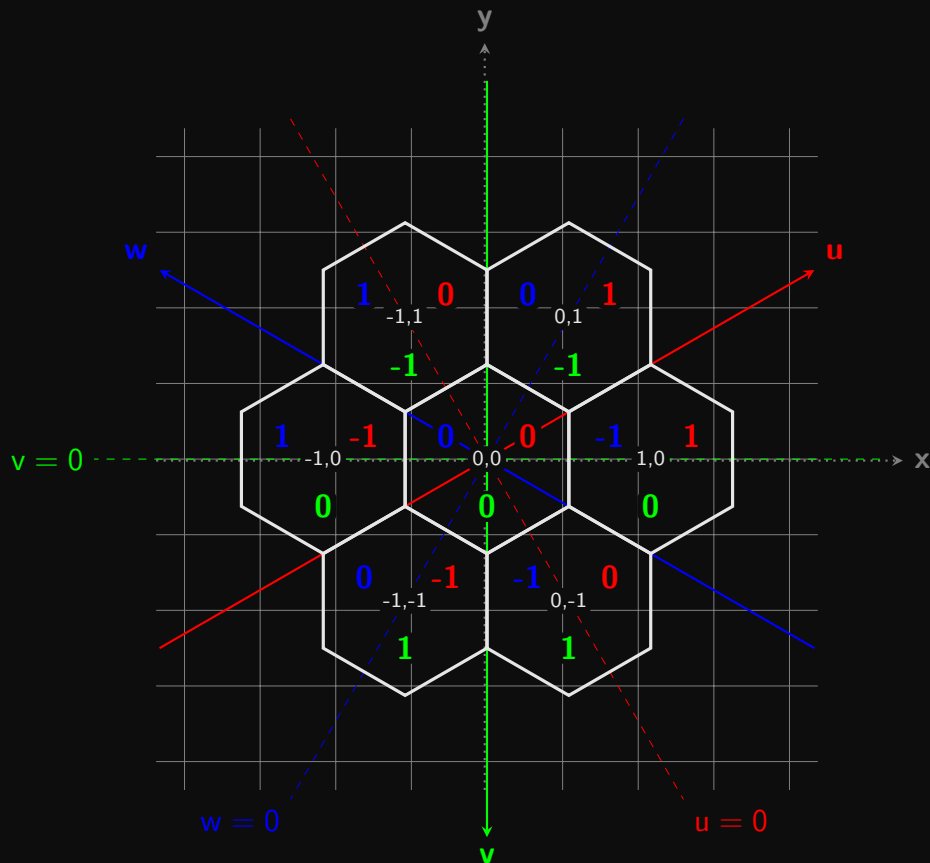


Figure 42: Hexagonal Coordinate System

The diagram shows the u , v and w axes that form the hexagonal coordinate system as well as the grid represents the regular cartesian coordinate system with x and y axes. Additionally, the white coordinates in the center of each hexagon display their offset coordinates.

Due to their homogeneity, all of these hexagonal coordinates lie in a diagonal plane through the origin of a three-dimensional space. This simplifies algorithms and allows the reuse of existing ones, that are designed for cube grids. The coordinate system is based on a comprehensive blog article about *Hexagonal Grids*¹, which should be referred to for further details.

¹ Patel, [Thoughts on Grids](#); Patel, [Hexagonal Grids](#).

Coordinate Conversion

Based on an edge length of 1, the horizontal distance between adjacent pointy hexagons is $\sqrt{3}$ (see section 4.3.1 Tiles). The vertical distance is equal to $\frac{3}{2}$ if every second row is shifted to the right by $\frac{1}{2}\sqrt{3}$. This results in the following conversion functions between cartesian, hexagonal and offset coordinates:

The shift for **offset** coordinates depends on the system handedness:

$$\mathbf{N} \rightarrow \mathbf{N} : \quad \mathbf{f}_v(v) = \begin{cases} -v & \text{if right-handed} \\ v & \text{if left-handed} \end{cases}$$

A right-handed offset coordinate system is used in the implementation, which means the x-axis points east and the y-axis north.

Converting **cartesian** coordinates to **hexagonal** coordinates:

$$\mathbf{N}^2 \rightarrow \mathbf{N}^3 : \quad \mathbf{f}_{ch}(x, y) = \begin{pmatrix} \frac{\sqrt{3} \cdot x - \mathbf{f}_v(y)}{3} \\ \frac{2}{3} \cdot \mathbf{f}_v(y) \\ 0 - u - v \end{pmatrix} = \begin{pmatrix} u \\ v \\ w \end{pmatrix}$$

Converting **hexagonal** coordinates to **offset** coordinates:

$$\mathbf{N}^3 \rightarrow \mathbf{N}^2 : \quad \mathbf{f}_{ho}(u, v, w) = \begin{pmatrix} u + \frac{1}{2} \cdot v \\ \mathbf{f}_v(v) \end{pmatrix} = \begin{pmatrix} c \\ r \end{pmatrix}$$

Converting **offset** coordinates to **cartesian** coordinates:

$$\mathbf{N}^2 \rightarrow \mathbf{N}^2 : \quad \mathbf{f}_{oc}(c, r) = \begin{pmatrix} \sqrt{3} \cdot c \\ \frac{3}{2} \cdot r \end{pmatrix} = \begin{pmatrix} x \\ y \end{pmatrix}$$

These conversions are necessary to convert tile coordinates into world positions which can then be used to sample the generated property maps. They are also utilized in ray intersection calculations that are required for mouse based tile selection.

4.4 Rendering

4.4.1 User Interface

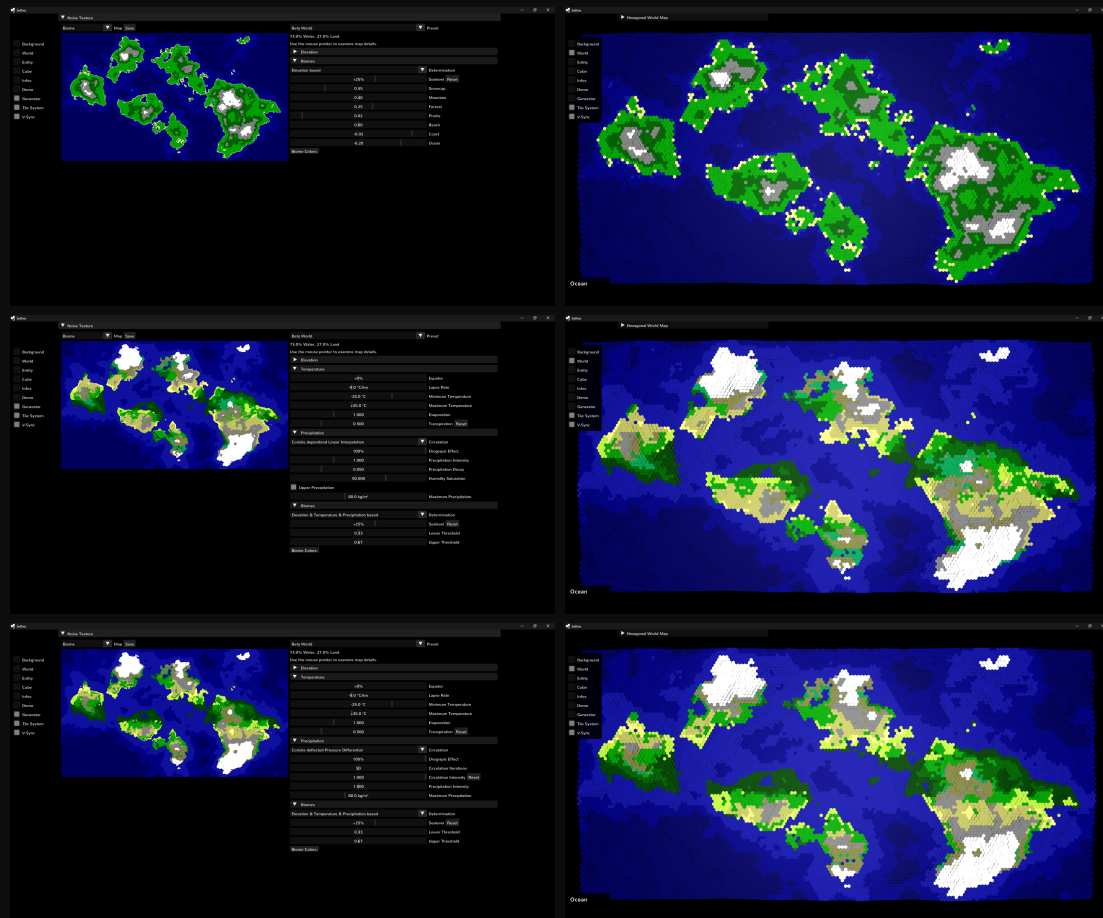


Figure 43: User Interface of the World Generator

The graphical user interface of the world generator is realized in `C++` using `Cinder` with `OpenGL` and `ImGui`. The application programming interface of the latter was enhanced with custom comfort functions to further streamline the user code, resulting in minimal implementation effort for property controls such as:

```
1 update |= ui::SliderFloat(label, variable, minimum, maximum, display, power, reset);
```

Listing 9: Property Controls

This instruction shows a value slider according to the placement constraints set in its current scope.

update is a boolean that is turned into *true* if the slider value changes, otherwise it remains unmodified.

label is a string that contains the text, displayed to the left of the slider.

variable is the float variable that changes with the slider value.

minimum is a float value that specifies the slider's lower limit.

maximum is a float value that specifies the slider's upper limit.

display is a string that specifies display format for the slider value.

power is a float value that specifies the value range distribution on the slider.

reset is a float value that specifies the default value.

(A reset button is displayed next to the slider if its value deviates from the reset value.)

Immediate Interface Instructions

These instructions can be used anywhere in the executed code, hence the name *immediate mode graphical user interface*. Based on them, *ImGui* builds vertex buffers and a small list of draw call batches, which are then efficiently rendered using *OpenGL*.

Incremental Generation Process

The generation process works incrementally (see figure 22). This means changing properties that affect a certain step requires updating of all subsequent steps, but the results of the previous ones are still valid. Therefore, the output textures of every step are kept in memory during the entire runtime of the program to allow their reuse. Once a step is recomputed, its output texture is overwritten.

Dynamic Biome Sampling

Closing the generator panel causes the resulting biome map to be sampled for every hexagon tile of the world map. This returns a color that is then mapped to a biome type, represented by an enumeration value. Modifying the map resolution changes the number of tiles and triggers sampling again (see figure 44).

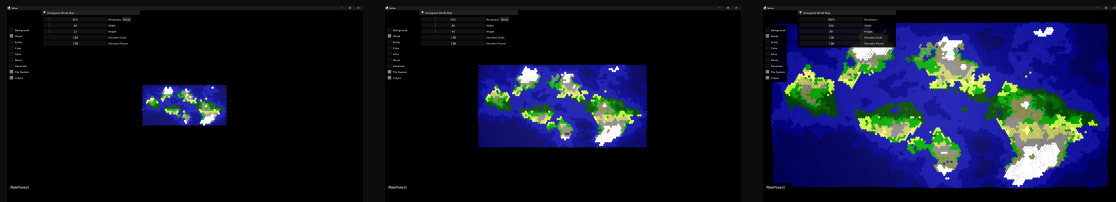


Figure 44: Different World Resolutions

From left to right: 25%, 50%, 100% of a 160 × 80 tile world resolution.

The world generation process is not executed again unless the generator panel is opened again and its properties are modified.

4.4.2 Tile Instantiation

The hexagon tiles are implemented as entities using a custom *entity component system*. Apart from an *id*, these tile entities only possess two properties: *hexagonal coordinates* and a *biome type*. They use a shared geometry with a shared material and are rendered using Instantiation at positions defined in a dedicated vertex buffer. These positions can be updated to allow continuous scrolling of the horizontally wrapping map.

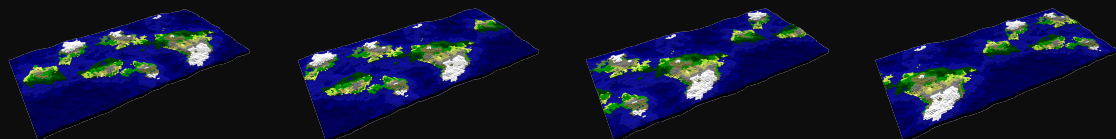


Figure 45: Wrapping Hexagon Tile based World Map

5 Conclusion

This section evaluates the result 5.1, summarizes the thesis 5.2 and gives an outlook on possible improvements 5.3 of the presented methods.

5.1 Result

To evaluate the presented methods, topography and bathymetry maps of the Earth are used to compose an elevation map and start the world generation process (see section 4.2). The resulting biome maps can then be compared to a reference image of the Earth:

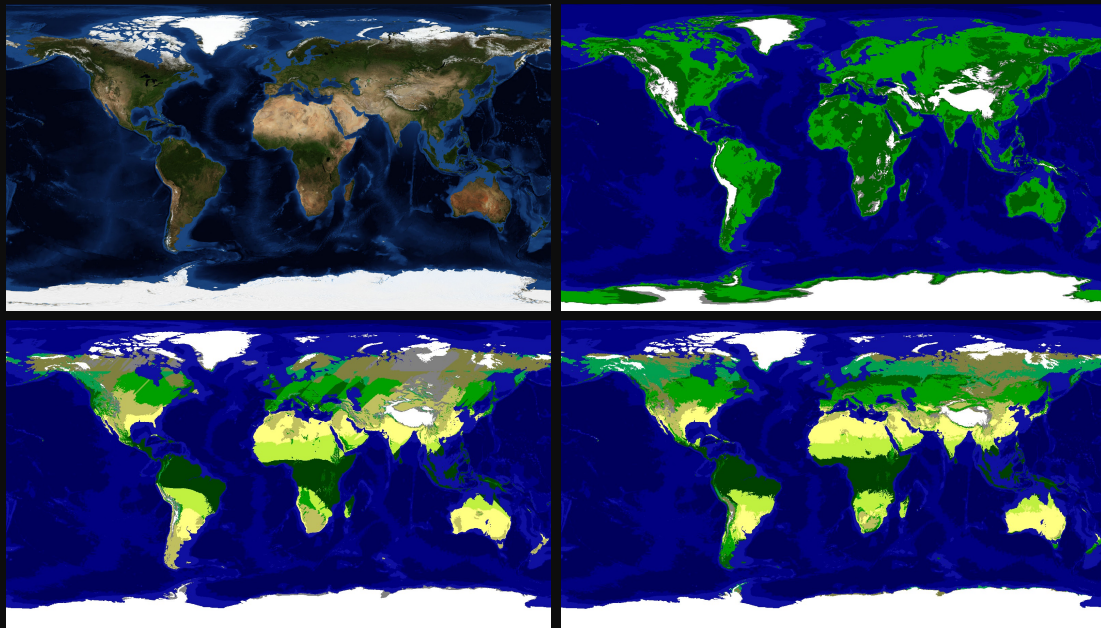


Figure 46: Comparison of Biome Generation.

Reference image of the Earth (upper left), elevation based biomes (upper right) and biomes using approximated linear circulation (lower left) or pressure based circulation (lower right).

NASA, *Visible Earth: Blue Marble*, 2004

The elevation based map shows the problems described in section 3.4, namely repetitive patterns and a lack in biome diversity, whereas the results of the climate based methods show similarities to the real world. Apparently, the approximated linear circulation method seems to better replicate the dry climate in Central Asia than the more advanced pressure based circulation method, but it suffers from the linear artifacts described in section 3.5.1. The pressure based method does not suffer from these artifacts and resembles the climate in Europe, Southeast Asia and New Zealand more accurately.

Regarding the separation of land and water, especially the lake in Middle Australia, using *flood fill* in combination with *hydraulic erosion* would yield more realistic results.

5.2 Summary

This thesis presented a method to generate world maps with pseudo-realistic biomes at runtime using essential climate principles. After describing the fundamental concepts of Earth's climate, the limitations of purely random based biome generation were discussed. Then, a simple method that approximates global circulation using a linear humidity convection was introduced. Finally, a pressure based circulation method was presented and its implementation explained in detail. Additionally, the ideas behind a hexagon tile based world map, that was used to showcase the results, were addressed.

A video summarizing the ideas behind this thesis is available on the Internet.¹

5.3 Outlook

Since this thesis focused on fast execution speed, there are many possibilities to improve the realism of the generated world maps by reducing this constraint and incorporating more sophisticated simulations (see section 2.3 [Similar Works](#)). The most important ones would be [plate tectonics](#) and [hydraulic erosion](#), which could produce natural looking mountain chains, rivers and lakes (see section 3.3 [Simulation based Terrain Generation](#)). Further possible improvements include the emulation of local effects like coastal wind patterns and migrating pressure areas.

¹ Tenjix, [Climate based Biomes](#), 2016.

Affidavit

I hereby declare that this thesis is entirely the result of my own work except where indicated otherwise. All excerpts from other sources have been marked clearly and are listed as references.

A handwritten signature in black ink, appearing to be 'M. W.' followed by a flourish.

Signature

Weingarten, 2017-02-28

Place & Date

Appendix

Utility Functions

```
1 // maps unsigned range [0,1] to signed range [-1,+1]
2 float to_signed_range(float value) {
3     return (value - 0.5) * 2.0;
4 }
```

Listing 10: Value Conversion: Unsigned to Signed

```
1 // maps signed range [-1,+1] to unsigned range [0,1]
2 float to_unsigned_range(float value) {
3     return value * 0.5 + 0.5;
4 }
```

Listing 11: Value Conversion: Signed to Unsigned

```
1 // creates a two-dimensional counter-clockwise rotation matrix
2 mat2 rotation_matrix(float angle) {
3     float sine = sin(angle), cosine = cos(angle);
4     return mat2(cosine, -sine, sine, cosine);
5 }
```

Listing 12: Two-Dimensional Rotation Matrix

Acronyms

CPU *central processing unit* 28, 30

FBM *Fractional Brownian Motion* 17

GPU *graphics processing unit* 28, 30

NASA North American Space Agency , 64

PCG *procedural content generation* 6

RAM *random-access memory*

TPU *tensor processing unit*

Glossary

C++

C++ is a general-purpose programming language with a bias towards systems programming that supports data abstraction, object-oriented programming, generic programming and functional programming. ISO, *C++ Programming Language* 28, 45, 52, 54

central processing unit

A central processing unit (CPU) is the electronic circuitry within a computer that carries out the instructions of a computer program by performing the basic arithmetic, logical, control and input/output operations specified by the instructions. Wikipedia, *Central Processing Unit* 28, 51

Cinder

Cinder is a free and open source library for professional-quality creative coding in C++. *Cinder* 28, 45

coherent noise

Coherent noise is a smooth form of random noise that fulfills certain criteria. (see section 3.2.1) 16, 30

Coriolis Effect

This effect causes the deflection of moving fluids and gases on both hemispheres of a rotating planet. (see section 2.4.3) 8, 11, 23, 25, 34, 36

dew point

The dew point is the temperature to which air must be cooled to become saturated with water vapor. When further cooled, the airborne water vapor will condense to form liquid dew. Wikipedia, *Dew Point* 11, 12

entity component system

An entity component system is an architectural pattern that follows the composition over inheritance principle that allows greater flexibility in defining entities. Every entity consists of one or more components which add additional behavior or functionality. Wikipedia, *Entity Component System* 28, 46

equatorial plane

The equatorial plane is a plain that passes through the center of a planet and contains its geometric equator as a circle. What Is, [Latitude and Longitude](#) , 55

evaporation

Evaporation is a type of vaporization of a liquid that occurs from the surface of a liquid into a gaseous phase that is not saturated with the evaporating substance.

Wikipedia, [Evaporation](#) 38, 53, 54, 58

evapotranspiration

Evapotranspiration is the sum of water [evaporation](#) and plant [transpiration](#) from the Earth's land and ocean surface to the atmosphere. Wikipedia, [Evapotranspiration](#) 25, 34, 38, 39

Extra Credits

Extra Credits is a video lesson series that discuss issues pertinent to video games and game studies, particularly discussing issues concerning video game development, addressing the legitimacy of video games as art, and creating intellectual discourse on important issues in gaming culture. Wikipedia, [Extra Credits](#) 6

flood fill

Flood fill, also called seed fill, is an algorithm that determines the area connected to a given node in a multi-dimensional array. The algorithm looks for all nodes in the array that are connected to the start node by a path of the target color and changes them to the replacement color. Wikipedia, [Flood Fill](#) 32, 47

Fractional Brownian Motion

Fractional Brownian Motion (FBM), is a generalization of Brownian Motion. Unlike the classical version, its increments need not be independent. (see section [3.2.2](#))

Wikipedia, [Fractional Brownian Motion](#) 16, 30, 51

global strategy game

The term global strategy game or 4X stands for a genre of strategy-based video and board games in which players control an empire to "eXplore, eXpand, eXploit, and eXterminate". Wikipedia, [4X](#) 4

graphics processing unit

A graphics processing unit (GPU) is a specialized electronic circuit designed to rapidly manipulate and alter memory to accelerate the creation of images in a frame buffer intended for output to a display device. Wikipedia, [Graphics Processing Unit](#) 28, 51

Holdridge Life Zones

Holdridge Life Zones provide a global bioclimatic scheme for the classification of land areas based on precipitation, [evaporation](#) and temperature. It was first published by Leslie Holdridge in 1947, and updated in 1967. Wikipedia, [Holdrige Life Zones](#) 8

hydraulic erosion

Hydraulic erosion refers to water based erosion. Erosion is the action of surface processes that remove soil, rock or dissolved material from one location on the Earth's crust and transport it to another location. (see section 3.3) Wikipedia, [Erosion](#) 20, 32, 47, 48

hydrology

Hydrology is the scientific study of the movement, distribution, and quality of water on Earth and other planets, including the water cycle, water resources and environmental watershed sustainability. Wikipedia, [Hydrology](#) 9

ImGui

ImGui (immediate mode graphical user interface), is a bloat-free graphical user interface library for [C++](#). It outputs optimized vertex buffers that you can render anytime in your 3D-pipeline enabled application. It is fast, portable, renderer agnostic and self-contained (no external dependencies). Cornut, [ImGui](#) 28, 45, 46

inertial force

The inertial force, also called fictitious force, is a pseudo force that acts on all masses whose motion is described using a non-inertial frame of reference, such as a rotating reference frame. Wikipedia, [Fictitious Force](#) 13

infinite series

In mathematics, the infinite series is an elementary example of a geometric series that converges absolutely. Its sum is $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 1$. Wikipedia, [Infinite Series](#) 17

intertropical convergence zone

The Intertropical Convergence Zone, known by sailors as the doldrums, is the area encircling the earth near the equator where the northeast and southeast trade winds come together. Wikipedia, [Intertropical Convergence Zone](#) 11, 38

jet stream

Jet streams are fast flowing, narrow, meandering air currents found in the atmosphere of some planets, including Earth. Wikipedia, [Jet Stream](#) 11

lapse rate

The lapse rate is the rate at which atmospheric temperature decreases with an increase in altitude. Wikipedia, [Lapse Rate](#) 11, 12, 58

latitude

Latitude is the angular deviation of a planet's surface point from its [equatorial plane](#). It's defined as the angle between the [equatorial plane](#) and a line that passes through the planet's center and the surface point. Latitude angles range from +90 degrees (north) to -90 degrees (south). What Is, [Latitude and Longitude](#) 8, 10, 11, 19, 23, 27, 33, 38, 56

longitude

Longitude is the angular deviation of a planet's surface point from its [prime meridian](#). It's defined as the angle between the [prime meridian](#) and a [meridian](#) that passes through the surface point. Longitude angles range from +180 degrees (east) to -180 degrees (west). What Is, [Latitude and Longitude](#)

meridian

A meridian is a half-circle that runs from one geometric pole of a planet to the other. What Is, [Latitude and Longitude](#) , 55, 56

midpoint displacement

Midpoint displacement, usually done with the diamond-square algorithm, is a fast method to generate height maps by subdividing rectangles and randomly displacing the height of their corners. Procedural Content Generation Wiki, [Midpoint Displacement Algorithm](#) 8

OpenGL

OpenGL stands for Open Graphics Library. It is a specification of an API for rendering graphics, usually in 3D. OpenGL implementations are libraries that implement the API defined by the specification. Graphics cards usually have an OpenGL implementation. Khronos, [Open Graphics Library](#) 28, 45, 46

Orographic Effect

This effect causes wet windward slopes and dry leeward slopes as well as dry areas in the rain-shadow of mountains. (see section 2.4.2) 8, 23, 25, 34, 39, 56, 57

orographic precipitation

Orographic precipitation occurs when moist air rises over mountains and causes wet slopes on the windward side due to the [Orographic Effect](#). 8

Perlin Noise

Perlin noise is a type of gradient noise developed by Ken Perlin in 1983 as a result of his frustration with the "machine-like" look of computer graphics at the time. Wikipedia, [Perlin Noise](#) 8, 30, 57

permutation table

A permutation table is a list of randomly permuted numbers containing all consecutive numbers within a certain range. 4

plate tectonics

Plate tectonics describes the motion of the large rigid plates in the outermost shell of a planet (the crust and upper mantle). Where the plates meet, their relative motion determines the type of boundary: convergent, divergent, or transform. (see section 3.3) Wikipedia, [Plate Tectonics](#) 20, 48

prime meridian

The prime [meridian](#) has to be arbitrarily defined and acts as reference to define [latitudes](#). What Is, [Latitude and Longitude](#) , 55

procedural content generation

Procedural content generation is the algorithmic generation of content for games or simulations using a random or pseudo-random process that results in varying content based on underlying parameters. It is one aspect of [procedural generation](#) and usually involves the usage of a [pseudo-random number generator](#). 4, 7–9, 51

procedural generation

In computing, procedural generation is a method of creating data algorithmically as opposed to manually. In computer graphics, it is also called random generation and is commonly used to create textures and 3D models. In video games it is used to automatically create large amounts of content in a game. Advantages of procedural generation include smaller file sizes, larger amounts of content, and randomness for less predictable gameplay. Wikipedia, [Procedural Generation](#) 7, 56

pseudo-random number generator

A pseudo-random number generator, also known as a deterministic random bit generator, is an algorithm for generating a sequence of numbers whose properties approximate the properties of sequences of random numbers. The generated sequence is not truly random, because it is completely determined by a relatively small set of initial values, called seed. Although sequences that are closer to truly random can be generated using hardware random number generators, pseudo-random number generators are important in practice for their speed in number generation and their reproducibility. Wikipedia, [Pseudorandom Number Generator](#) 4, 16, 56

rain shadow

A rain shadow describes an area with a lack of precipitation on the leeward side of mountains, caused by the [Orographic Effect](#). 8, 26

random-access memory

Random-access memory (RAM) is a form of computer data storage which stores frequently used program instructions to increase the general speed of a system. A random-access memory device allows data items to be read or written in almost the same amount of time irrespective of the physical location of data inside the memory. Wikipedia, [Random-access Memory](#) 51

Simplex Noise

Simplex noise is better looking, faster and has proper derivatives without the grid artifacts of [Perlin Noise](#) derivatives. Geilfus, [Simplex Noise](#) Ken Perlin designed the algorithm in 2001. Wikipedia, [Simplex Noise](#) 28, 30

stratosphere

The stratosphere is the second major layer of Earth's atmosphere, just above the [troposphere](#), and below the mesosphere. About 20% of the atmosphere's mass is contained in the stratosphere. The stratosphere is stratified in temperature, with

warmer layers higher and cooler layers closer to the Earth. Wikipedia, [Stratosphere](#) , 58

suspension of disbelief

The term suspension of disbelief or willing suspension of disbelief has been defined as a willingness to suspend one's critical faculties and believe the unbelievable; sacrifice of realism and logic for the sake of enjoyment. Wikipedia, [Suspension Of Disbelief](#) 7

tensor processing unit

A tensor processing unit (TPU) is an application-specific integrated circuit developed specifically for machine learning. Compared to graphics processing units, they are designed explicitly for a higher volume of reduced precision computation and lack hardware for rasterization and texture mapping. Wikipedia, [Tensor Processing Unit](#) 51

terrain synthesis

Terrain synthesis is a process in which procedurally generated content is merged with real-world terrain data using various feature analysis and composing techniques. Zhou et al., "[Terrain Synthesis from Digital Elevation Models](#)", 2007 7

transpiration

Transpiration is the process of water movement through a plant and its [evaporation](#) from aerial parts, such as leaves, stems and flowers. Wikipedia, [Transpiration](#) 38, 53

tropopause

The tropopause is the region of the atmosphere where the environmental [lapse rate](#) changes from positive, as it behaves in the [troposphere](#), to the stratospheric negative one in the [stratosphere](#). Wikipedia, [Tropopause](#) 11

troposphere

The troposphere is the lowest portion of Earth's atmosphere where all weather takes place. It contains approximately 75% of the atmosphere's mass and 99% of the total mass of water vapor and aerosols. The average depths of the troposphere are 20 km in the tropics, 17 km in the mid latitudes, and 7 km in the polar regions in winter. Wikipedia, [Troposphere](#) 11, 57, 58

uncanny valley

In aesthetics, the uncanny valley is the hypothesis that human replicas which appear almost, but not exactly, like real human beings elicit feelings of eeriness and revulsion among some observers. Valley denotes a dip in the human observer's affinity for the replica, a relation that otherwise increases with the replica's human likeness. Wikipedia, [Uncanny Valley](#) 4, 7

WebGL

WebGL stands for Web Graphics Library. It is a cross-platform, royalty-free web standard for a low-level 3D graphics API based on OpenGL ES, exposed to ECMAScript via the HTML5 Canvas element. WebGL brings plugin-free 3D to the web, implemented right into the browser. Khronos, [Web Graphics Library](#) 30

Figures

1	Earth's Global Atmospheric Circulation	10
2	Orographic Effect causes Rain-Shadows	12
3	Coriolis Effect on Northern (left) and Southern (right) Hemisphere	12
4	Earth's Global Vegetation Areas	13
5	Biomes based on Precipitation and Temperature	14
6	Procedural Noise based Terrain	16
7	Fractal Coherent Noise	17
8	Comparison of Noise based Terrain	18
9	Plate Tectonics	20
10	Hydraulic Erosion	20
11	Elevation based Thresholds	21
12	Purely Elevation based Biomes	22
13	Purely Elevation based World	22
14	Approximated linear Circulation	23
15	Climate based Biomes using Approximated linear Circulation	24
16	Climate based World using Approximated linear Circulation	24
17	Pressure based Circulation	25
18	Climate based Biomes using Pressure based Circulation	26
19	Climate based World using Pressure based Circulation	26
20	Hexagon Tile based World Map with 80 x 45 Tiles.	27
21	Hexagon Tile based World Map with 160 x 90 Tiles.	27
22	Overview of the Biome Generation Steps	29
23	Elevation Map, generated with Simplex Noise	30
24	Elevation Map, composed from Bathymetry and Topography	31
25	Biome Map based purely on Elevation	32
26	Temperature Map	33
27	Moving Equator	33
28	Precipitation Map using Approximated linear Circulation	34
29	Surface Wind Directions using Approximated linear Circulation.	34
30	Biome Map using Approximated linear Circulation	35
31	Circulation Map	36
32	Surface Air Pressure and Flow of Pressure based Circulation	37
33	Components of the Circulation Map showing Surface Air Pressure and Flow	37
34	Evapotranspiration Map	38
35	Humidity Map	39
36	Humidity Convection	39
37	Precipitation Map	40
38	Biome Map using Pressure based Circulation	41
39	Hexagon Tile	42
40	Hexagon Tile Types	42
41	Hexagon Map Movement	42

Listings

1	Fractal Noise	30
2	Elevation based Biome Determination	32
3	Normalized Distance to Equator	33
4	Climate based Biome Determination	35
5	Coriolis Effect	36
6	Moisture Estimation	38
7	Orographic Effect	39
8	Circulation based Precipitation	40
9	Property Controls	45
10	Value Conversion: Unsigned to Signed	50
11	Value Conversion: Signed to Unsigned	50
12	Two-Dimensional Rotation Maxtix	50

References

Ahrens, C. Donald

Meteorology today: an introduction to weather, climate, and the environment. 9th ed. Brooks/Cole, Cengage Learning Belmont, 2009 (cit. on pp. 11–13).

Amplitude Studios

Endless Legend. 2014. URL: <http://www.amplitude-studios.com/endless-legend> (visited on 2017-02-28) (cit. on pp. 9, 27).

Ashima Arts

Simplex Noise. URL: <https://github.com/ashima/webgl-noise> (visited on 2017-02-28) (cit. on p. 30).

Bergensten, Jens (Jeb)

Minecraft World Generator Update. Mojang. 2013-08-28. URL: <https://mojang.com/2013/08/minecraft-world-generator-update> (visited on 2017-02-28) (cit. on p. 9).

Boesch, Florian

WebGL GPU Landscaping and Erosion. 2011. URL: <http://codeflow.org/entries/2011/nov/10/webgl-gpu-landscaping-and-erosion/> (visited on 2017-02-28) (cit. on p. 20).

Cepero, Miguel

Procedural World: Intelligent Terrain Synthesis. Voxel Farm Dev-Blog: Procedural World. 2016-07-15. URL: <http://procworld.blogspot.de/2016/07/intelligent-terrain-synthesis.html> (visited on 2017-02-28) (cit. on p. 7).

Procedural World: Uncanny Valley of Procedural Generation. Voxel Farm Dev-Blog: Procedural World. 2012-07-09. URL: <http://procworld.blogspot.de/2012/07/uncanny-valley-of-procedural-generation.html> (visited on 2017-02-28) (cit. on p. 7).

Voxel Farm Dev-Blog: Procedural World. URL: <http://procworld.blogspot.de> (visited on 2017-02-28) (cit. on p. 7).

Cinder. URL: <https://libcinder.org/> (visited on 2017-02-28) (cit. on p. 52).

Community, Minecraft

Minecraft Version History. URL: http://minecraft.gamepedia.com/Version_history (visited on 2017-02-28) (cit. on p. 9).

Cornut, Omar

ImGui. URL: <https://github.com/ocornut/imgui> (visited on 2017-02-28) (cit. on p. 54).

Dahlskog, Steve and Julian Togelius

“Patterns and Procedural Content Generation: Revisiting Mario in World 1 Level 1”. In: *Proceedings of the First Workshop on Design Patterns in Games*. DPG '12. <http://doi.acm.org/10.1145/2427116.2427117>. Raleigh, North Carolina, USA: ACM, 2012, 1:1–1:8. ISBN: 978-1-4503-1854-9. DOI: 10.1145/2427116.2427117. URL: <http://julian.togelius.com/Dahlskog2012Patterns.pdf> (visited on 2017-02-28) (cit. on p. 6).

Extra Credits

Procedural Generation - How Games Create Infinite Worlds. 2015-07-22. URL: <https://www.youtube.com/watch?v=TgbuWfGeG2o> (visited on 2017-02-28) (cit. on p. 6).

Frampton, Dave

Ambience Devblog: Climate. 2015. URL: https://www.youtube.com/watch?v=j1FMhT3kz-c&list=PLv_YAI_PeTtf_qNdtiVjaz0HN9hEw8AxP&index=11 (visited on 2017-02-28) (cit. on p. 8).

Gallant, Jon

Procedurally Generating Wrapping World Maps. 2016-01-07. URL: <http://www.jgallant.com/procedurally-generating-wrapping-world-maps-in-unity-csharp-part-1/> (visited on 2017-02-28) (cit. on pp. 19, 22).

Geilfus, Simon

Simplex Noise. URL: <https://github.com/simongeilfus/SimplexNoise> (visited on 2017-02-28) (cit. on pp. 30, 57).

Harris, John

Interview: The Making of Dwarf Fortress. Gamasutra. 2008-02-27. URL: http://www.gamasutra.com/view/feature/3549/interview_the_making_of_dwarf_.php?page=7 (visited on 2017-02-28) (cit. on pp. 8, 13).

Hendrikx, Mark et al.

"Procedural Content Generation for Games: A Survey". In: *ACM Transactions on Multimedia Computing, Communications, and Applications* 9.1 (2013-02), 1:1–1:22. ISSN: 1551-6857. DOI: 10.1145/2422956.2422957. URL: <http://www.ds.ewi.tudelft.nl/~iosup/pcg-g-survey11tomccap-cr.pdf> (visited on 2017-02-28) (cit. on p. 6).

Khronos

Open Graphics Library. URL: <https://www.khronos.org/opengl/> (visited on 2017-02-28) (cit. on p. 56).

Web Graphics Library. URL: <https://www.khronos.org/webgl/> (visited on 2017-02-28) (cit. on p. 59).

Lague, Sebastian

Procedural Landmass Generation. 2016. URL: <https://www.youtube.com/watch?v=wbpMiKiSKm8> (visited on 2017-02-28) (cit. on p. 17).

LibNoise

What is coherent noise? URL: <http://libnoise.sourceforge.net/coherentnoise/index.html> (visited on 2017-02-28) (cit. on p. 16).

Mattson, Mark P.

"Superior pattern processing is the essence of the evolved human brain". In: *Frontiers in Neuroscience* 8 (2014). PubMed Central, p. 265. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC4141622/> (visited on 2017-02-28) (cit. on pp. 4, 7).

Mei, Xing, Philippe Decaudin, and Baogang Hu

"Fast Hydraulic Erosion Simulation and Visualization on GPU". In: *Pacific Graphics*. 2007. URL: <http://evasion.imag.fr/Publications/2007/MDH07> (visited on 2017-02-28) (cit. on p. 20).

Met Office

Global circulation patterns. 2016. URL: <http://www.metoffice.gov.uk/learning/learn-about-the-weather/how-weather-works/global-circulation-patterns> (visited on 2017-02-28) (cit. on p. 10).

Mindwerks

Diving Into Procedural Content Generation, With WorldEngine. 2016-03-28. URL: <https://www.smashingmagazine.com/2016/03/procedural-content-generation-introduction> (visited on 2017-02-28) (cit. on p. 8).

World Engine. 2014. URL: <http://world-engine.org/> (visited on 2017-02-28) (cit. on pp. 8, 20).

North American Space Agency (NASA)

Visible Earth: Blue Marble. 2004. URL: http://visibleearth.nasa.gov/view_cat.php?categoryID=1484 (visited on 2017-02-28) (cit. on pp. 31, 47).

Patel, Amit

Hexagonal Grids. Red Blob Games. URL: <http://www.redblobgames.com/grids/hexagons> (visited on 2017-02-28) (cit. on p. 43).

Thoughts on Grids. Red Blob Games. URL: <http://www-cs-students.stanford.edu/~amitp/game-programming/grids/> (visited on 2017-02-28) (cit. on p. 43).

Perlin, Ken

"Noise Hardware". In: *SIGGRAPH 2002 Course 36 Notes: Real-Time Shading Languages* (2002). URL: <https://www.csee.umbc.edu/~olano/s2002c36/ch02.pdf> (visited on 2017-02-28) (cit. on p. 30).

Persson, Markus (Notch)

The World of Notch: Terrain Generation. 2011-03-09. URL: <http://notch.tumblr.com/post/3746989361/terrain-generation-part-1> (visited on 2017-02-28) (cit. on p. 9).

Procedural Content Generation Wiki

Games Featuring Procedural Generation. URL: <http://pcg.wikidot.com/category-pcg-games> (visited on 2017-02-28) (cit. on p. 4).

Midpoint Displacement Algorithm. URL: <http://pcg.wikidot.com/pcg-algorithm:midpoint-displacement-algorithm> (visited on 2017-02-28) (cit. on p. 55).

Quílez, Íñigo

Fractional Brownian Motion Derivatives. URL: <http://www.iquilezles.org/www/articles/morenoise/morenoise.htm> (visited on 2017-02-28) (cit. on p. 17).

Ricklefs, Robert E.

The Economy of Nature. 6th ed. W.H. Freeman & Co Ltd, 2008 (cit. on pp. 13, 14).

Rouse, Margaret

Latitude and Longitude. TechTarget (WhatIs.com). URL: <http://whatIs.techtarget.com/definition/latitude-and-longitude> (visited on 2017-02-28) (cit. on pp. 53, 55, 56).

Shaker, Noor, Julian Togelius, and Mark J. Nelson

Procedural Content Generation in Games: A Textbook and an Overview of Current Research. Springer, 2016. URL: <http://pcgbook.com/> (visited on 2017-02-28) (cit. on p. 16).

Stroustrup, Bjarne

C++ Programming Language. International Organization for Standardization (ISO). URL: <https://isocpp.org/std> (visited on 2017-02-28) (cit. on p. 52).

Voxel Farm

Voxel Farm - Voxel Engine and Procedural Generation. 2014. URL: <http://www.voxelfarm.com/index.html> (visited on 2017-02-28) (cit. on p. 7).

Voyager Games

Outer Colony Dev-Blog: Game Overview. URL: <https://voyagergames.com/tfr-overview/> (visited on 2017-02-28) (cit. on p. 9).

Outer Colony Dev-Blog: Procedural World Generation. 2015. URL: <https://voyagergames.com/procedural-world-generation/> (visited on 2017-02-28) (cit. on p. 9).

Voyager Games

Outer Colony Dev-Blog: World Simulation. URL: <https://voyagergames.com/world-simulation/> (visited on 2017-02-28) (cit. on p. 9).

West, Mick

"Practical Fluid Mechanics Part I". In: *Game Developer Magazine* (2007-03). <http://cowboyprogramming.com/2008/04/01/practical-fluid-mechanics/>, pp. 43–47. URL: http://twvideo01.ubm-us.net/o1/vault/GD-Mag-Archives/GDM_March_2007.pdf (visited on 2017-02-28) (cit. on p. 25).

"Practical Fluid Mechanics Part II". In: *Game Developer Magazine* (2007-04). <http://cowboyprogramming.com/2008/04/01/practical-fluid-mechanics/>, pp. 30–32. URL: http://twvideo01.ubm-us.net/o1/vault/GD-Mag-Archives/GDM_April_2007.pdf (visited on 2017-02-28) (cit. on p. 25).

Whittaker, Robert H.

"Classification of Natural Communities". In: *Botanical Review* 28.1 (1962), pp. 1–239. ISSN: 00068101, 18749372. URL: <http://www.jstor.org/stable/4353649> (visited on 2017-02-28) (cit. on p. 14).

Wikipedia

4X. URL: <https://en.wikipedia.org/wiki/4X> (visited on 2017-02-28) (cit. on p. 53).

Atmospheric Circulation. URL: https://en.wikipedia.org/wiki/Atmospheric_circulation (visited on 2017-02-28) (cit. on p. 10).

Central Processing Unit. URL: https://en.wikipedia.org/wiki/Central_processing_unit (visited on 2017-02-28) (cit. on p. 52).

Coriolis Effect. 2007. URL: https://commons.wikimedia.org/wiki/File:Coriolis_effect10.svg (visited on 2017-02-28) (cit. on p. 12).

Dew Point. URL: https://en.wikipedia.org/wiki/Dew_point (visited on 2017-02-28) (cit. on p. 52).

Dwarf Fortress. URL: https://en.wikipedia.org/wiki/Dwarf_Fortress (visited on 2017-02-28) (cit. on p. 8).

Earth's Global Circulation. 2013. URL: https://en.wikipedia.org/wiki/File:Earth_Global_Circulation_-_en.svg (visited on 2017-02-28) (cit. on p. 10).

Earth's Global Vegetation. 2016. URL: <https://en.wikipedia.org/wiki/File:Vegetation.png> (visited on 2017-02-28) (cit. on p. 13).

Entity Component System. URL: <https://en.wikipedia.org/wiki/Entity%E2%80%93component%E2%80%93system> (visited on 2017-02-28) (cit. on p. 52).

Erosion. URL: <https://en.wikipedia.org/wiki/Erosion> (visited on 2017-02-28) (cit. on p. 54).

Evaporation. URL: <https://en.wikipedia.org/wiki/Evaporation> (visited on 2017-02-28) (cit. on p. 53).

Evapotranspiration. URL: <https://en.wikipedia.org/wiki/Evapotranspiration> (visited on 2017-02-28) (cit. on p. 53).

Extra Credits. URL: https://en.wikipedia.org/wiki/Extra_Credits (visited on 2017-02-28) (cit. on p. 53).

Fictitious Force. URL: https://en.wikipedia.org/wiki/Fictitious_force (visited on 2017-02-28) (cit. on p. 54).

Flood Fill. URL: https://en.wikipedia.org/wiki/Flood_fill (visited on 2017-02-28) (cit. on p. 53).

Wikipedia

Fractional Brownian Motion. URL: https://en.wikipedia.org/wiki/Fractional_Brownian_motion (visited on 2017-02-28) (cit. on p. 53).

George Hadley. URL: https://en.wikipedia.org/wiki/George_Hadley (visited on 2017-02-28) (cit. on p. 10).

Graphics Processing Unit. URL: https://en.wikipedia.org/wiki/Graphics_processing_unit (visited on 2017-02-28) (cit. on p. 54).

Holdrige Life Zones. URL: https://en.wikipedia.org/wiki/Holdridge_life_zones (visited on 2017-02-28) (cit. on p. 54).

Hydrology. URL: <https://en.wikipedia.org/wiki/Hydrology> (visited on 2017-02-28) (cit. on p. 54).

Infinite Series. URL: https://en.wikipedia.org/wiki/1/2_%2B_1/4_%2B_1/8_%2B_1/16_%2B_%E2%8B%AF (visited on 2017-02-28) (cit. on p. 54).

Intertropical Convergence Zone. URL: https://en.wikipedia.org/wiki/Intertropical_Convergence_Zone (visited on 2017-02-28) (cit. on p. 55).

Jet Stream. URL: https://en.wikipedia.org/wiki/Jet_stream (visited on 2017-02-28) (cit. on p. 55).

Lapse Rate. URL: https://en.wikipedia.org/wiki/Lapse_rate (visited on 2017-02-28) (cit. on p. 55).

Perlin Noise. URL: https://en.wikipedia.org/wiki/Perlin_noise (visited on 2017-02-28) (cit. on p. 56).

Plate Tectonics. URL: https://en.wikipedia.org/wiki/Plate_tectonics (visited on 2017-02-28) (cit. on pp. 20, 56).

Procedural Generation. URL: https://en.wikipedia.org/wiki/Procedural_generation (visited on 2017-02-28) (cit. on p. 57).

Pseudorandom Number Generator. URL: https://en.wikipedia.org/wiki/Pseudorandom_number_generator (visited on 2017-02-28) (cit. on p. 57).

Rain Shadow. 2007. URL: https://commons.wikimedia.org/wiki/File:Rain_shadow_no_text.svg (visited on 2017-02-28) (cit. on p. 12).

Random-access Memory. URL: https://en.wikipedia.org/wiki/Random-access_memory (visited on 2017-02-28) (cit. on p. 57).

Simplex Noise. URL: https://en.wikipedia.org/wiki/Simplex_noise (visited on 2017-02-28) (cit. on p. 57).

Stratosphere. URL: <https://en.wikipedia.org/wiki/Stratosphere> (visited on 2017-02-28) (cit. on p. 58).

Suspension Of Disbelief. URL: https://en.wikipedia.org/wiki/Suspension_of_disbelief (visited on 2017-02-28) (cit. on p. 58).

Tensor Processing Unit. URL: https://de.wikipedia.org/wiki/Tensor_Processing_Unit (visited on 2017-02-28) (cit. on p. 58).

Transpiration. URL: <https://en.wikipedia.org/wiki/Transpiration> (visited on 2017-02-28) (cit. on p. 58).

Tropopause. URL: <https://en.wikipedia.org/wiki/Tropopause> (visited on 2017-02-28) (cit. on p. 58).

Wikipedia

Troposphere. URL: <https://en.wikipedia.org/wiki/Troposphere> (visited on 2017-02-28) (cit. on p. 58).

Uncanny Valley. URL: https://en.wikipedia.org/wiki/Uncanny_valley (visited on 2017-02-28) (cit. on p. 59).

William Ferrel. URL: https://en.wikipedia.org/wiki/William_Ferrel (visited on 2017-02-28) (cit. on p. 10).

Würstle, Thomas (Tenjix)

Climate based Biomes. 2016. URL: <https://tenjix.de/projects/climate-based-biomes> (visited on 2017-02-28) (cit. on p. 48).

Zhou, Howard et al.

“Terrain Synthesis from Digital Elevation Models”. In: *IEEE Transactions on Visualization and Computer Graphics* 13.4 (2007-07), pp. 834–848. URL: http://www.cc.gatech.edu/~turk/my_papers/terrain_synth_tvlg.pdf (visited on 2017-02-28) (cit. on pp. 7, 58).